

MCP Tunnels

How they work, why your firewall never sees them, what telemetry actually exists, how to prevent them — and a reproducible EC2 and Windows lab.

AUDIENCE	AI security architects, detection engineers, red and blue teams in regulated financial services
SCOPE	Copilot / agentic tooling estates using MCP for data access and money movement
PROTOCOL BASELINE	MCP 2026-07-28 (current), with 2025-11-25 and 2025-06-18 compatibility notes
LAB TARGETS	Amazon Linux 2023 / Ubuntu 24.04 on EC2; Windows 10/11 or Server 2022 with PowerShell 5.1+
POSTURE	Defensive. Every procedure runs against infrastructure you own.
VERSION	1.0 — September 2026

Contents

1. How to use this guide	4
1.1 What this answers	4
1.2 Prerequisites and rules of engagement	4
2. Threat model: copilots and tooling in a bank	6
2.1 What actually changed	6
2.2 Surface 1 — money movement and beneficiary tooling	6
2.3 Surface 2 — confidentiality and the data boundary	7
2.4 Surface 3 — citizen-built custom copilots	8
2.5 Reference frameworks worth citing internally	8
3. MCP transports: what is actually on the wire	10
3.1 stdio	10
3.2 Streamable HTTP	10
3.3 What the 2026-07-28 stateless core changed	10
3.4 Annotated request, with the detection value of each field	11
4. What "MCP tunnel" means — a working taxonomy	13
4.1 The five types	13
4.2 T1 — stdio-to-HTTP bridges	13
4.3 T2 — reverse ingress tunnels	14
4.4 T3 — shadow MCP	14
4.5 T4 — MCP as a covert channel	15
4.6 T5 — governed-path bypass	15
5. Why there are no inbound ports	16
5.1 The classic model your controls were built for	16
5.2 The inversion, step by step	16
5.3 Why each existing control fails	17
5.4 Provider-specific mechanics and ports	17
5.5 Where control actually moves to	18
6. Detection: the log sources that exist	20
6.1 Coverage matrix	20
6.2 Client layer	21
6.3 Host layer	22
6.4 Network layer	23
6.5 Identity layer	23
6.6 Server layer	24
6.7 Blind spots you must write down	24
7. Detection content you can deploy	25
7.1 Sigma — tunnel agent execution	25
7.2 Sigma — MCP client config tampering	26
7.3 KQL — Defender / Sentinel	27
7.4 Splunk SPL	28
7.5 Zeek and network analytics	29
7.6 Linux auditd and osquery	29
7.7 The high-fidelity correlation	30
7.8 Tool-description drift detection	31
8. Prevention: what actually holds	32
8.1 Control tiers	32
8.2 Tier 1 — binary and execution control	32

8.3 Tier 1 — egress architecture	33
8.4 Tier 1 — the governed MCP path	34
8.5 Tier 2 — identity and authorization	35
8.6 Tier 2 — money-movement specific controls	35
8.7 Cloud and EC2 specifics	36
8.8 Copilot Studio and low-code specifics	37
9. The lab: reproduce it end to end	38
9.0 Topology and build	38
Step 1 — Baseline and prove the negative	38
Step 2 — Stand up an MCP server on loopback	39
Step 3 — Confirm it is unreachable	40
Step 4 — Arm telemetry before you tunnel	40
Step 5 — Open the tunnel	41
Step 6 — Prove the inversion	41
Step 7 — Read the telemetry, find the gap	42
Step 8 — Add inspection, recover the signal	43
Step 9 — Test the preventive controls	44
Step 10 — The Windows run	45
Step 11 — Teardown and artifact checklist	46
10. Control matrix	48
11. Assessment checklist for the copilot programme	50
12. References	52

1. How to use this guide

1.1 What this answers

You are assessing a copilot and tooling estate that reaches MCP servers to do beneficiary work, fund transfers and similar operations, and you need a defensible position on MCP tunnels specifically: what they are, what detects them, what logs exist, and whether prevention is even possible given there is no inbound port to filter.

The short answer to the last question, expanded in [Section 5: the absence of an inbound port is real, not a misunderstanding](#). A reverse tunnel makes a service internet-reachable by riding an outbound-established TCP session. Nothing in your ingress path ever sees a new inbound flow, so nothing in your ingress path can block it. Prevention is entirely possible, but every control that works sits somewhere other than the inbound ACL: process execution control, egress architecture, DNS, identity, and server-side authorization.

Sections 2 through 8 are the analysis. Section 9 is a lab you can run in an afternoon on one EC2 instance and one Windows host that demonstrates each claim with commands and observable output. Sections 10 and 11 are the artifacts you hand to a control owner or an auditor.

1.2 Prerequisites and rules of engagement

Item	Detail
Cloud lab	One EC2 instance, <code>t3.small</code> or larger. Amazon Linux 2023 or Ubuntu 24.04. Non-production VPC and account. Public subnet with an Elastic IP so you can demonstrate the ingress negative.
Security group	Inbound: TCP 22 from your own address only. Nothing else, ever. Outbound: start fully open, then restrict in Step 9 — the contrast is the lesson.
Windows lab	Windows 10/11 or Server 2022 VM. Sysmon installed with a config that logs Event ID 1, 3 and 22. PowerShell 5.1 or 7.
Packages	Linux: <code>python3</code> , <code>tcpdump</code> , <code>iproute2</code> , <code>audit</code> , <code>bind-utils</code> / <code>dnsutils</code> , <code>jq</code> . Optional: <code>mitmproxy</code> , <code>zeek</code> .
Tunnel client	<code>cloudflared</code> . Quick Tunnels need no account, which keeps the lab self-contained. <code>ngrok</code> works identically if you prefer; the free tier needs a token.
Accounts	Use a personal or sandbox tunnel account. Do not authenticate a tunnel agent with corporate credentials, even in a lab.

RULES OF ENGAGEMENT

Everything here runs against systems you own and data you invented. The lab MCP server ships with synthetic accounts and its write tool performs no write. Do not point a tunnel at a corporate host, do not expose a real internal service, and do not run the hunt scripts against production endpoints without the same change approval you would need for any endpoint-touching tool. If your organisation restricts tunnel binaries, get the exception in writing before you download `cloudflared` — an unannounced tunnel from a security engineer's workstation looks exactly like the thing you are writing detections for, and that is a bad day for everyone.

A note on terminology

"MCP tunnel" is not a term the specification defines, and people use it for at least five distinct things. Treating them as one thing is why control conversations stall — the detection and the fix are different in each case. [Section 4](#) separates them into T1 through T5 and the rest of the guide uses those labels.

2. Threat model: copilots and tooling in a bank

2.1 What actually changed

Enterprise permission models were designed around two assumptions about the human holding the entitlement: that they exercise judgement when something looks wrong, and that they act at human speed. A payments analyst with authority to add a beneficiary can add perhaps a few dozen in a day, and will stop if the request feels off.

An agent inherits the entitlement and breaks both assumptions. Its decisions are non-deterministic, and it can invoke the same tool thousands of times without fatigue. A plausible but wrong decision becomes a plausible but wrong decision executed at API rate. This framing is Cloudflare's and it is the clearest one-paragraph statement of the problem I have seen; it is worth reusing verbatim in your risk write-up.

The NSA's May 2026 Cybersecurity Information Sheet on MCP adds the protocol-level version of the same point. MCP inverts the familiar interaction pattern: rather than clients requesting data from servers, MCP frequently expects servers to query and execute actions on behalf of connected clients. That inversion creates attack paths that existing controls were never asked to trace. The CSI names financial services explicitly as a sector where MCP is already in production for sensitive tasks.

THE CONDITION TO TEST FOR

An agent becomes dangerous when three properties hold simultaneously: it can **access sensitive data**, it is **exposed to untrusted content**, and it has an **outbound channel**. Any two are usually survivable. All three means a single successful injection chains into exfiltration or unauthorised action. Every agent review in your programme should answer these three questions before anything else, and the answer to at least one of them should be no for any agent that can move money.

2.2 Surface 1 — money movement and beneficiary tooling

This is the surface that turns an AI governance problem into a fraud-loss problem, and it deserves disproportionate attention in your assessment.

The mechanics

A tool such as `add_beneficiary` or `initiate_transfer` exposed over MCP is an API call with a natural-language trigger. The agent decides to call it based on the model's interpretation of context. That context includes anything the agent has read: an email, a PDF invoice, a ticket comment, a CRM note, a supplier portal page. Content the bank does not control is now an input to a control decision the bank does not log at the semantic layer.

The attack does not need to defeat authentication. It rides a legitimate session, with a legitimately entitled identity, invoking a legitimately registered tool with attacker-chosen arguments. Every individual event in your existing logs looks authorised, which is precisely why the OWASP MCP Top 10 lists *Lack of Audit and Telemetry* (MCP08) as a top-ten risk in its own right rather than a supporting concern.

Specific failure modes to test

Failure mode	What it looks like in your estate
Injected beneficiary change	A supplier invoice PDF or inbound email carries instructions the agent reads as operator intent. The agent calls the beneficiary tool with attacker-supplied bank details. The analyst sees a clean summary. This is the EchoLeak pattern (CVE-2025-32711) with a write tool on the end of it instead of a read.
Dual-control collapse	Maker-checker was designed around two humans in two sessions. If the agent performs the maker step and a human rubber-stamps a generated summary as the checker, you have one point of failure wearing two hats. Verify where the second control is actually enforced: in the core banking system, or in the agent's workflow definition?
Confused deputy across the tool boundary	The MCP server holds broader privilege than the calling user and does not bind each request to a specific client identity and approved operation. A user who can reach the server can reach everything the server can reach. Server-level access control is necessary and nowhere near sufficient.
Token passthrough	The MCP server accepts a token it did not validate as issued to itself and forwards it downstream. The core banking API then trusts it as though the MCP server had vouched for it. The specification explicitly forbids this, and implementations still do it — Apollo's own MCP Server was documented forwarding the caller's validated token upstream by default as recently as August 2026.
Idempotency gaps	MCP does not enforce idempotency; the NSA CSI calls this out directly. A retried or replayed <code>tools/call</code> against a transfer endpoint without a server-side idempotency key is a duplicate payment.
Scope creep over time	A tool approved for read access to balances gains a write capability in a later server version. If tool metadata is applied without a re-approval step, the change is live within a polling interval and nobody was asked. OWASP MCP02.

The question to ask the tool owner

Not "is the agent authenticated." Ask: *if the model is fully compromised by content it read, what is the worst single sequence of tool calls it can execute before a human sees anything?* If the answer includes a completed outbound payment, the control is in the wrong place. The control belongs at the tool handler and at the core system, not in the prompt.

2.3 Surface 2 — confidentiality and the data boundary

Three distinct leakage paths, each with a different owner:

- **Egress in tool arguments.** The arguments to a `tools/call` are the most sensitive part of an MCP request. They carry the query, the customer record, the source code, the document being summarised. If the destination is an unapproved server, the payload has left the bank inside an ordinary-looking HTTPS POST. This is the primary loss channel for shadow MCP (T3) and it is invisible without TLS inspection.
- **Context blending across tasks.** The NSA CSI flags shared and overlapping context windows as a systemic risk: multiple MCP servers connected to the same client are exposed to information the client shares freely, with no isolation guarantees in the protocol. In a bank this maps directly onto a segregation-of-duties problem — a research tool and a customer-data tool on the same agent means the research tool's operator can potentially see customer data.

- **Response-side leakage.** Tool *results* also cross the boundary and are equally worth inspecting. NSA guidance is explicit: treat every tool output as untrusted input to the next stage, filter it, and log it before it is passed on.

ALIGNMENT WITH DATA CLASSIFICATION

The NSA's practical recommendation is to align tools and models with data classification zones: public tools grouped to handle public data, and tools touching regulated financial information explicitly segregated and controlled. For private data it recommends a local MCP server instance over a remote one. In a bank this translates into a hard rule worth writing into standard: **no agent may hold, in a single context, both a tool that reads regulated customer data and a tool that can reach an uncontrolled external destination.**

2.4 Surface 3 — citizen-built custom copilots

Low-code agent builders let a business user assemble an agent in an afternoon: pick knowledge sources, attach connectors or MCP servers, publish. The security posture of that agent is the product of three things the maker chose — topics, tools and knowledge sources — and misconfiguration in any one of them can expose data or enable unauthorised action.

The recurring problems, which you should build into an agent intake review:

- **User-credential tool auth.** Makers frequently attach tools that authenticate with their own credentials. The agent then acts with the maker's entitlements for every user who invokes it.
- **Generic HTTP connectors.** A generic HTTP action is an arbitrary egress channel wearing a connector's clothing, and on some platforms it bypasses tenant isolation policy by design. Treat it as the single highest-risk item in any agent definition.
- **Generative orchestration.** When the model chooses which tool to call rather than following a deterministic flow, the tool-selection decision itself becomes injectable. For any agent with a write tool touching money or customer records, deterministic topics are the safer default and the burden of proof should sit with whoever wants generative orchestration.
- **Invisible telemetry.** Platform-native prompt-injection shields often emit events that never reach the SOC unless an integration is explicitly configured. Verify the pipeline exists; do not assume the vendor's dashboard is wired to your SIEM.
- **No inventory.** If you cannot enumerate every agent, its tools, its knowledge sources and its MCP servers on demand, you do not have a programme. This is OWASP MCP09 in low-code form.

2.5 Reference frameworks worth citing internally

Three sources carry enough weight to anchor a control standard, and using them saves you from arguing your own taxonomy in a governance forum.

Source	Use it for
NSA AISC CSI U/OO/6030316-26, May 2026	Fifteen pages of prescriptive baseline from a national authority, developed with CMU SEI. Names the design gaps (access control, serialization, approval workflows, token lifecycle, audit) and gives concrete recommendations including filtering outbound proxies, sandboxing, message signing and network scanning for rogue servers. The single best document to attach to a policy exception request.

Source	Use it for
OWASP MCP Top 10 2025 edition	Shared vocabulary for findings. MCP01 Token Mismanagement; MCP02 Privilege Escalation via Scope Creep; MCP03 Tool Poisoning; MCP04 Supply Chain; MCP05 Command Injection; MCP06 Intent Flow Subversion; MCP07 Insufficient AuthN/AuthZ; MCP08 Lack of Audit and Telemetry; MCP09 Shadow MCP Servers; MCP10 Context Injection and Over-Sharing.
MCP specification 2026-07-28 security best practices	Normative language you can hold a vendor to. Confused deputy mitigation, the explicit prohibition on token passthrough, and session-handling requirements are all stated with MUST .

3. MCP transports: what is actually on the wire

You cannot write detections for a protocol you have not looked at. This section is short and entirely concerned with observable artifacts.

The current specification defines exactly two transports. The legacy HTTP+SSE transport from 2024-11-05 is formally deprecated as of 2026-07-28 with a year-long offramp, so you will still meet it in the field through most of 2027.

3.1 stdio

The server runs as a local subprocess of the client. Messages are newline-delimited JSON-RPC over the child process's stdin and stdout. That is the entire transport.

THE CONSEQUENCE YOU MUST INTERNALISE

A stdio MCP server generates **no network traffic, opens no socket, and binds no port**. There is no packet for a proxy, a secure web gateway, a firewall, a flow log or an IDS to see. Every network-layer MCP control on the market, including Cloudflare's protocol detection, states this limitation explicitly. If your entire MCP visibility strategy is network-based, your coverage of local servers is zero, and local servers are the majority of installed MCP servers.

Coverage for stdio comes from exactly three places: the client (hooks and managed policy), the host (process telemetry, EDR, auditd), and the filesystem (client configuration files). Section 6 covers all three.

3.2 Streamable HTTP

Introduced in 2025-03-26. The server runs independently, exposes a single MCP endpoint path supporting both POST and GET, and may optionally use Server-Sent Events to stream server-to-client messages. This is the transport that made remote MCP deployments practical and it is the one you can see on the wire.

Two things trip up detection engineers here. First, **the endpoint path is not standardised**. `/mcp` is a convention, not a requirement. A perfectly conformant MCP server can live at `https://tools.example.com/api`. Second, **the hostname carries no signal either**. Cloudflare's own first-generation detection searched Gateway logs for hostnames containing `mcp` and paths like `/mcp` or `/sse`, and their published assessment of that approach is that it is very basic: it misses servers at ordinary URLs and it produces false positives on unrelated services. Useful for historical hunting; not a control.

3.3 What the 2026-07-28 stateless core changed

This release is the largest revision since launch and it is unambiguously good news for defenders, because it moved the protocol's identity out of connection state and onto every individual request.

Change	Detection and control impact
<code>initialize</code> / <code>initialized</code> handshake retired; <code>Mcp-Session-Id</code> gone (SEP-2575, SEP-2567)	No more first-request blind spot where the opening exchange lacked the version header. Every request now carries its own protocol version, client identity and capabilities. Session-hijacking risk tied to a guessable or stealable session ID largely evaporates, and so does the session-resumption abuse path.

Change	Detection and control impact
Mcp-Method and Mcp-Name headers now required on Streamable HTTP (SEP-2243)	The single most useful change. A gateway, WAF or rate limiter can now see the operation and the tool name without parsing the JSON body. You can rate-limit <code>tools/call</code> separately from <code>tools/list</code> , and you can write a policy on a specific tool name. Servers reject requests where the headers and body disagree — which is itself a tamper signal worth alerting on.
MCP-Protocol-Version required on every POST	This is the header Cloudflare Gateway's <code>experimental.is_mcp</code> selector keys on. Presence is a strong positive indicator. Absence proves nothing.
MRTR replaces server-initiated <code>elicitation/create</code> , <code>sampling/createMessage</code> , <code>roots/list</code> (SEP-2322)	No held-open bidirectional stream needed for mid-call user input. Fewer long-lived connections means less traffic that looks like a persistent channel — and conversely, a long-lived stream in a 2026-07-28 environment is now more anomalous than it used to be.
List results carry <code>ttlMs</code> and <code>cacheScope</code> (SEP-2549)	Clients cache tool catalogues. For you, this means a client may be operating on a cached tool list; a tool-poisoning change may not be re-fetched immediately. Factor the TTL into your drift-detection interval.
W3C Trace Context in <code>_meta</code> (SEP-414) — <code>traceparent</code> , <code>tracestate</code> , <code>baggage</code>	Key names are now fixed in the spec. A trace that starts in the host application can follow a tool call through the client SDK and the gateway. If you own the stack, this is your correlation ID across the whole agentic chain. Instrument it.
Dynamic Client Registration deprecated in favour of CIMD; RFC 9207 iss validation required; client credentials bound to issuer (SEP-2468, SEP-2352)	DCR was the mechanism behind the classic MCP confused-deputy attack. Its deprecation, plus issuer validation, closes the authorization-server mix-up hole. For a bank this is the change that makes pre-registered OAuth clients the sanctioned pattern — which is what your IAM team wanted anyway.
Roots, Sampling and Logging deprecated (SEP-2577); legacy HTTP+SSE deprecated	Twelve-month minimum support window. Inventory anything using them now; they are a migration ticket, not an emergency.

3.4 Annotated request, with the detection value of each field

A conformant 2026-07-28 tool call. Compare this against what your gateway logs actually retain.

HTTP — 2026-07-28 STATELESS TOOL CALL

```
POST /mcp HTTP/1.1
Host: tools.example.com
Authorization: Bearer <access-token>
Content-Type: application/json
MCP-Protocol-Version: 2026-07-28
Mcp-Method: tools/call
Mcp-Name: add_beneficiary

{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "tools/call",
  "params": {
    "name": "add_beneficiary",
    "arguments": {
      "account_id": "ACCT-1001",
      "beneficiary_name": "NORTHWIND SUPPLY LLC",
      "account_number": "0044179912"
    }
  },
  "_meta": {
    "io.modelcontextprotocol/clientInfo": {"name": "copilot-desktop", "version": "3.2"},

```

```
"traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01"
}
```

Field	What it gives you
Host + path	Destination. Approved server or not. Available from SNI even without decryption; the path is not.
Authorization	Presence, and with decryption the token's audience claim. An aud that does not match the server identifier is a token-passthrough finding.
MCP-Protocol-Version	This request is MCP. The classification primitive.
Mcp-Method	Read vs. write vs. discovery, without body parsing. tools/call is the one that changes state.
Mcp-Name	The tool being invoked. This is what lets you write "block add_beneficiary from any source other than the approved portal" as a network policy rather than an application change.
params.arguments	The payload. Account numbers, PII, source code, queries. Requires body inspection. This is what DLP needs to see and what you must never log raw — hash it, as the lab server does.
_meta.traceparent	Correlation across client, gateway and server. Free if you configure it.

LEGACY COMPARISON

A 2025-11-25 client sends MCP-Protocol-Version but not the routing headers, so you get "this is MCP" without "which tool". A pre-2025-06-18 client sends neither and is invisible to header-based classification entirely. When you scope a detection project, get the client version distribution first — it determines what fidelity you can actually reach.

4. What "MCP tunnel" means — a working taxonomy

4.1 The five types

Five different things get called an MCP tunnel. They have different mechanics, different telemetry and different fixes. Use the T-labels consistently in findings and your control conversations get shorter.

Type	Mechanism	Who initiates TCP	Exposure created	Primary control
T1	stdio↔HTTP bridge (<code>mcp-remote</code> , <code>supergateway</code> , <code>mcp-proxy</code>)	Local process → internet, outbound	A stdio-only client reaches a remote server. Credentials and arguments leave the host.	Package control, egress proxy, client policy
T2	Reverse ingress tunnel (<code>cloudflared</code> , <code>ngrok</code> , Tailscale Funnel, dev tunnels)	Local agent → provider edge, outbound	Inbound reachability from the public internet with no inbound firewall rule. The headline case.	Binary allowlisting, DNS, egress allowlist
T3	Direct client→remote MCP ("shadow MCP")	Client → third-party server, outbound	Data egress in tool arguments to an unvetted destination; supply-chain and tool-poisoning exposure.	TLS-inspecting SWG with MCP classification; MCP portal
T4	MCP used as the transport for a covert channel	Either	C2 or exfiltration hidden inside legitimate-looking tool calls.	Behavioural analytics on tool-call sequences; server-side audit
T5	Governed-path bypass	Client → approved server, direct	An <i>approved</i> server reached directly, skipping the portal's identity, tool catalogue, DLP and audit.	Origin-side rejection of non-portal traffic

4.2 T1 — stdio-to-HTTP bridges

Most MCP clients started life stdio-only. Bridges exist to let those clients talk to remote servers: the client spawns the bridge as a local stdio subprocess, and the bridge makes the outbound HTTPS connection and handles OAuth. `mcp-remote` is the canonical example and had over 437,000 weekly downloads at the time of its disclosure.

Why this matters beyond the obvious

A bridge is a supply-chain component sitting directly in the credential path. CVE-2025-6514 (CVSS 9.6, JFrog, July 2025) is the case study: a malicious or compromised remote MCP server returns a crafted `authorization_endpoint` URL during the OAuth flow, and `mcp-remote` passes it unsanitised to the operating system, giving arbitrary OS command execution on the client machine. Versions 0.0.5 through 0.1.15 are affected; 0.1.16 fixes it. It was the first documented case of full RCE on a client OS purely from connecting to an untrusted MCP server.

The structural lesson outlives the patch. A bridge inverts the trust assumption that made stdio safe. Locally installed stdio servers were implicitly trusted because the user chose to run them; a bridge extends that implicit trust to whatever is at the far end of a URL.

Artifacts to hunt

- Process command lines containing `mcp-remote`, `supergateway`, `mcp-proxy`, `mcpo`, `stdio-to-sse`, typically launched via `npm` or `uvx` as a child of an editor or desktop client.
- `npm` as a parent of a long-lived `node` process holding an outbound TLS session — unusual outside CI.
- MCP client config files with an `args` array whose first element is a bridge package name and whose second is a URL.
- Package-manager telemetry: an install of a bridge package on an endpoint that is not a build agent.

4.3 T2 — reverse ingress tunnels

This is what most people mean by "MCP tunnel" and it is the subject of Section 5. The pattern is documented as the standard developer workflow by multiple AI vendors: run an MCP server on localhost, run a tunnel agent beside it, paste the resulting public URL into the assistant's connector configuration. It is not obscure, it is not an attack technique, and it is in the getting-started documentation. That is exactly why it appears in enterprises without anyone filing a change request.

The three ways it goes wrong

1. **Authentication is optional and often absent.** The tunnel provider gives you a public URL; whether anything behind it checks a credential is entirely up to the developer. A quick tunnel with an unauthenticated MCP server behind it is an open API for whoever finds the hostname. OWASP MCP09 cites analysis finding hundreds of MCP servers exposed to the internet.
2. **Subdomain reuse.** Free-tier tunnels issue a fresh random subdomain each run. If a tunnel stops and someone else later claims the same subdomain, requests from stale links — including OAuth redirects — reach a stranger. Persistent named subdomains and careful rotation are the mitigation.
3. **It survives the developer.** A tunnel started for a demo and left running as a background service becomes permanent internet exposure of an internal service, with no asset record and no owner. Akira ransomware operators used Cloudflare Quick Tunnels for exactly this persistence property; MAZE used ngrok before them. The technique predates MCP by years — MCP just gave it a legitimate-looking reason to be on a developer's machine.

4.4 T3 — shadow MCP

An employee finds a useful MCP server in a repository, a product guide or a colleague's message and adds it to their client. One line of configuration. No approval, no inventory entry, no idea what tools it exposes or what data leaves in the arguments.

OWASP tracks this as MCP09. The distinguishing property versus classic shadow IT is that the exposure is bidirectional: the organisation loses data through the tool arguments, and

simultaneously accepts the server's tool descriptions as instructions the model will follow. A shadow MCP server is both an exfiltration endpoint and an injection point.

Practical discovery layers, in rough order of coverage:

1. Static scan of repositories for `.mcp.json`, `mcp.json`, and MCP SDK dependencies in `package.json` / `pyproject.toml`.
2. Endpoint inspection of IDE and client config directories.
3. Network monitoring for outbound sessions from agent and editor processes.
4. Process monitoring for MCP subprocesses spawned by developer tools.
5. CI/CD audit — a server pulled only at build time is still shadow MCP.

4.5 T4 — MCP as a covert channel

Published research (arXiv 2511.15998, November 2025) proposes MCP as a C2 substrate and the reasoning is uncomfortable to read as a defender. MCP traffic rides standard TLS, is explicitly designed for event-driven rather than periodic communication, uses simple JSON, and looks indistinguishable from a modern AI-powered application synchronising state. The authors' claim is that the architecture eliminates the host and network artifacts — notably beacon periodicity — that traditional C2 detection depends on.

The related and more likely operational case is simpler: a malicious server participating in a legitimate agent workflow. Because it speaks the same protocol and mimics normal behaviour, the exfiltration appears in logs as an ordinary tool invocation.

DETECTION IMPLICATION

Everything you know about beacon detection — jitter analysis, interval clustering, connection-count baselining — degrades against a protocol that is legitimately bursty and event-driven. The signal moves up the stack. You are looking for *semantic* anomalies: a tool called at a volume or in a sequence that does not match its business purpose, arguments whose entropy or size is out of family, a read tool invoked immediately before an egress-capable tool. That analysis requires the server-side audit record described in Section 6.6. There is no network-only answer to T4.

4.6 T5 — governed-path bypass

Distinct from shadow MCP and frequently conflated with it. Here the server *is* approved and *is* behind a portal or gateway that enforces identity, a curated tool catalogue, DLP and tool-level audit. The employee configures their client against the upstream URL directly and skips all of it.

The traffic looks entirely legitimate: an approved destination, a valid credential, a normal tool call. Only the absence of the portal's fingerprint distinguishes it. Detecting T5 requires either a network control that can label traffic by on-ramp, or an origin that refuses requests which did not arrive through the sanctioned path. Cloudflare implements the first with a Traffic Source selector; the second is an Access policy, a source-IP restriction, or an enterprise authorization mechanism at the server. **You need both** — the network label only covers managed devices on managed paths.

5. Why there are no inbound ports

This is the core of the question and it deserves a precise answer, because the instinct that something is wrong here is correct. Your ingress controls are not failing to catch the tunnel. They are structurally incapable of seeing it.

5.1 The classic model your controls were built for

Publishing a service to the internet traditionally requires four things, each of which is a control point:

1. The process binds a socket to a **routable** address and listens.
2. A host firewall permits inbound on that port.
3. A network ACL, security group or edge firewall permits inbound on that port from the source range.
4. Something — DNAT, a load balancer, a public IP — maps an internet-reachable address to it.

Each step is an artifact. A listening socket on `0.0.0.0` shows in `ss -tlnp`. The firewall rule is in configuration management. The ACL change goes through your change process. The connection arrives as an inbound SYN from an external source and is logged as a new flow. This is the model behind the NSA's recommendation to periodically scan your networks for open or unauthorised MCP servers — sound advice that assumes the server is reachable at a routable address.

5.2 The inversion, step by step

A reverse tunnel deletes all four control points. Here is the actual sequence.

SEQUENCE

- [1] MCP server binds `127.0.0.1:8848`
 - loopback only. Not reachable from the NIC. Invisible to any external scan.
 - no host firewall rule required; loopback is not filtered
 - no security group change; no NACL change; no ticket
- [2] Tunnel agent starts on the same host
 - reads the local target from its own config or argv: `--url http://localhost:8848`
- [3] Tunnel agent opens an OUTBOUND connection to the provider edge
 - SYN leaves the host toward provider-edge:7844/udp (QUIC) or :443/tcp (TLS)
 - conntrack creates an ESTABLISHED entry for this 5-tuple
 - your egress policy sees: "an internal host made an outbound TLS connection". Allowed.
- [4] Provider assigns a public hostname and publishes DNS for it
 - `https://random-words-1234.trycloudflare.com`
 - this DNS record lives in the PROVIDER's zone, not yours. You do not see the change.
- [5] Someone on the internet requests that hostname
 - their TCP connection terminates at the PROVIDER's edge, not at your host
 - the provider multiplexes that request as a new stream INSIDE the session your host already established in step [3]
- [6] Your host receives the request as inbound bytes on an ESTABLISHED socket it opened
 - conntrack: matches an existing entry. Not a new inbound flow.
 - stateful firewall: return traffic on an established session. Permitted by definition.
 - tunnel agent forwards it to `127.0.0.1:8848` over loopback

[7] The MCP server answers. Response travels back the same path.

THE ONE SENTENCE TO PUT IN THE RISK WRITE-UP

A reverse tunnel converts an *ingress* problem, which your perimeter controls govern, into an *egress* problem, which most enterprises govern loosely. There is never a new inbound flow. Every packet carrying an internet user's request arrives on a socket your own host opened outbound, and is therefore indistinguishable at layers 3 and 4 from a response to an ordinary web request.

5.3 Why each existing control fails

Control	Effect	Why
AWS Security Group (inbound)	None	Security groups are stateful and only evaluate <i>new</i> inbound flows. The tunnel's traffic arrives as established-session return traffic. An SG with zero inbound rules permits it completely.
Network ACL (inbound)	None	NACLs are stateless and do evaluate return traffic, but the return traffic is on ephemeral ports from the provider's address, matching the allow rule that lets your outbound sessions work at all. Blocking it means breaking all outbound TCP.
Host firewall (<code>iptables INPUT</code> , Windows Defender Firewall inbound)	None	Same reason. Also: the MCP server is on loopback, and loopback traffic does not traverse the INPUT chain in the way you would need.
External vulnerability scan / port scan	None	The host has no open port. <code>nmap</code> against the Elastic IP returns nothing. The service is reachable only via a hostname in someone else's DNS zone.
Internal network scan for rogue MCP servers	Partial	Catches servers bound to routable addresses. Misses every loopback-bound server behind a tunnel. This is a real gap in the otherwise-sound NSA scanning recommendation and you should note it explicitly when you cite that guidance.
VPC Flow Logs	Weak	Records the outbound 5-tuple, bytes and packets. No hostname, no SNI, no URL, no headers. You see "host talked to a Cloudflare address on 7844". You cannot see that it published an MCP server.
NAT gateway / egress IP allowlisting	Weak	Tunnel providers run on large anycast ranges shared with enormous amounts of legitimate traffic. IP-based blocking is impractical; you need name-based or process-based control.
TLS-inspecting egress proxy	Strong	Sees the destination hostname and, with decryption, the HTTP semantics. This is where control actually lives. Note that QUIC will bypass an HTTP proxy unless UDP is blocked.
Application allowlisting	Strongest	The tunnel cannot start. Everything downstream becomes moot. This is the only control that addresses the root cause rather than the symptom.

5.4 Provider-specific mechanics and ports

You need these to write egress and DNS policy. Verify against current vendor documentation before you deploy — endpoints change.

Provider	Control-plane destinations	Ports	Notes for policy
cloudflared	<code>*.v2.argotunnel.com</code> , <code>protocol-v2.argotunnel.com</code> , <code>update.argotunnel.com</code> , <code>api.trycloudflare.com</code> , published as <code>*.trycloudflare.com</code>	UDP/7844 (QUIC, default), TCP/7844 (HTTP/2 fallback), TCP/443 (API)	Blocking UDP/7844 only forces HTTP/2 fallback — it does not stop the tunnel. Block both. Quick Tunnels need no account, which is why they are the highest-risk variant.
ngrok	<code>tunnel.{us,eu,ap,au,sa,jp,in}.ngrok.com</code> , <code>connect.ngrok-agent.com</code> , <code>bin.equinox.io</code> (download), published as <code>*.ngrok.io</code> / <code>*.ngrok-free.app</code>	TCP/443 TLS	Free tier injects an interstitial warning page unless the client sends <code>ngrok-skip-browser-warning</code> . That header is itself a hunting signal in proxy logs.
Tailscale Funnel	<code>controlplane.tailscale.com</code> , <code>derp*.tailscale.com</code> , published under <code>*.ts.net</code>	TCP/443, UDP/3478, UDP/41641	Often already sanctioned for remote access, which makes Funnel a quiet way to publish a service inside an approved tool. Audit Funnel usage separately from Tailscale usage.
VS Code / GitHub dev tunnels	<code>global.rel.tunnels.api.visualstudio.com</code> , <code>*.devtunnels.ms</code>	TCP/443	Built into an editor most developers already have. No separate binary to allowlist against, which makes it the hardest of the four to control by process.
localtunnel, serveo, bore, frp, pagekite	<code>*.loca.lt</code> , <code>serveo.net</code> , <code>*.lhr.life</code> , self-hosted <code>frps</code>	Various, incl. SSH/22	Self-hosted variants such as <code>frp</code> and plain <code>ssh -R</code> have no fixed provider domain. Only process control and egress allowlisting reach these.

DO NOT FORGET PLAIN SSH

`ssh -R 8080:localhost:8848 user@vps` is a reverse tunnel with no third-party provider, no distinctive domain and a binary that is present and sanctioned on essentially every host you own. Any policy that enumerates `cloudflared` and `ngrok` but ignores outbound SSH to non-corporate destinations has a hole you can drive through. On Windows, `plink.exe` and the built-in OpenSSH client are the equivalents.

5.5 Where control actually moves to

Five viable control points, in descending order of durability. Section 8 gives implementation detail for each.

- 1. Execution control on the endpoint.** If the tunnel binary cannot run, there is no tunnel. WDAC or AppLocker on Windows, `fail0cifyd` or SELinux on Linux.
- 2. Egress architecture.** Default-deny outbound, forced through a TLS-inspecting forward proxy, with UDP/443 and UDP/7844 blocked so QUIC cannot route around it.
- 3. DNS policy.** Block resolution of tunnel provider zones and log every attempt. Cheap, fast to deploy, and generates a clean high-signal alert; trivially evaded by an IP literal, so treat it as detection with a side of prevention.
- 4. Identity and authorization.** Even a reachable server should refuse unauthenticated callers, validate token audience, and reject requests that did not arrive via the sanctioned path.

5. **Server-side tool authorization.** The last point at which a request can be denied before the tool runs. Risk-tier every tool; block or require approval for write-critical ones regardless of how the request arrived.

6. Detection: the log sources that exist

6.1 Coverage matrix

Print this. It is the fastest way to answer "can we see X" in a control review, and the "does not give you" column is the one that prevents overclaiming.

Layer	Source	Gives you	Does <i>not</i> give you	Covers
Client	Client config files (<code>.mcp.json</code> , <code>claude_desktop_config.json</code> , <code>~/.cursor/mcp.json</code>)	Declared servers, transport, bridge args, credentials in plaintext. The inventory nothing else gives you.	Runtime behaviour. A config can exist unused, or a server can be added at runtime.	T1 T3 T5
	Client hooks / pre-tool-use callbacks	Destination, tool name and arguments <i>before</i> serialisation — no decryption needed. Covers stdio.	Anything on an unmanaged client. Requires reimplementation per client.	All
Host	Sysmon 1 / 3 / 22 / 23; EDR process telemetry	Tunnel and bridge execution with full command line, parent process, hashes, DNS queries.	Payload content. Nothing about which tool was called.	T1 T2
	Socket state (<code>ss</code> , <code>Get-NetTCPConnection</code>)	Loopback listeners and outbound established sessions with owning PID. The correlation input.	Point-in-time only unless you poll. Sysmon 3 excludes loopback by default.	T1 T2
	Linux auditd / eBPF	<code>execve</code> with <code>argv</code> , <code>connect()</code> , file writes to config paths. Kernel-level, hard to evade.	Encrypted payloads. Volume management is real work.	T1 T2
Network	DNS logs (Sysmon 22, Route 53 Resolver, resolver logs)	Tunnel provider lookups. Very high signal, very low volume, easy to deploy.	Nothing if an IP literal or DoH is used.	T2 T3
	TLS-inspecting SWG / forward proxy	Full URL, headers, body. <code>MCP-Protocol-Version</code> , <code>Mcp-Method</code> , <code>Mcp-Name</code> . DLP on arguments. The richest network source by a wide margin.	stdio. Off-network devices. Anything in a Do-Not-Inspect category. QUIC unless blocked.	T1 T2 T3 T5
	Zeek / NDR	<code>conn.log</code> duration and volume, <code>ssl.log</code> SNI and JA4 fingerprints, <code>dns.log</code> .	Anything inside TLS. SNI may be encrypted (ECH).	T2 T3
	VPC Flow Logs	5-tuple, bytes, packets, accept/reject. Long-lived high-volume flows.	No hostname, no SNI, no URL, no headers. Cannot identify MCP.	T2 (weak)
	AWS Network Firewall	TLS SNI-based rules and alert logs. Domain allow/deny that flow logs cannot do.	Payload without full TLS inspection.	T2 T3
Identity	IdP sign-in and OAuth consent grant logs	New client registrations, consent events, service principal creation, unusual token audiences.	Servers that use no auth at all — a large share of them.	T3 T5

Layer	Source	Gives you	Does <i>not</i> give you	Covers
	Token introspection at the resource	aud mismatches, which is your token-passthrough detector.	Requires the server to actually validate and log it.	T5
Server	MCP server audit log	Caller identity, tool name, argument digest, risk tier, outcome, source headers. The only source that sees semantics.	Servers you do not own. Most implementations emit nothing.	All
Platform	Low-code agent platform audit + DLP policy logs	Agent definitions, connector use, injection-shield events, agent inventory.	Frequently not wired to the SIEM by default.	T3 T4

6.2 Client layer

Client configuration files are the highest-value low-effort artifact in the whole set: they are a declared inventory, they are plaintext, and they frequently contain credentials in environment-variable blocks.

PATHS TO COLLECT — LINUX / MACOS

```
~/ .claude.json
~/ .claude/settings.json
~/ .claude/mcp.json
~/ .cursor/mcp.json
~/ .codeium/windsurf/mcp_config.json
~/ .config/Code/User/mcp.json
~/ .gemini/settings.json
/etc/claude-code/managed-settings.json      # enterprise-managed policy
<repo>/ .mcp.json                          # project-scoped: arrives with a git clone
```

PATHS TO COLLECT — WINDOWS

```
%USERPROFILE%\ .claude.json
%APPDATA%\Claude\claude_desktop_config.json
%USERPROFILE%\ .cursor\mcp.json
%APPDATA%\Code\User\mcp.json
%USERPROFILE%\ .codeium\windsurf\mcp_config.json
%ProgramData%\ClaudeCode\managed-settings.json
<repo>\ .mcp.json
```

PROJECT-SCOPED CONFIGS ARE THE SHARP EDGE

A `.mcp.json` committed to a repository means cloning that repository can register an MCP server in the developer's client. OX Security research disclosed in April 2026 (CVE-2026-30615 and related, spanning ten CVEs across MCP implementations) found all five major AI coding IDEs vulnerable to MCP-based prompt injection, with at least one version invoking the configured server command on repository open with **zero user interaction**. Treat `.mcp.json` in a repository as executable content: require review on change, and add it to your secret-scanning and pre-merge checks.

The other client-side control is the pre-tool-use hook. It runs after the model selects a tool but before the client serialises the request, so it sees the destination, tool name and arguments with no decryption — and critically, **it covers local studio servers**, which nothing on the network can. The

limitation is standardisation: you must reimplement per client, and telemetry from one client is never a complete inventory.

6.3 Host layer

Sysmon configuration that actually catches this

Two defaults will hurt you. First, **most Sysmon configurations exclude loopback in Event ID 3**, which is exactly where the tunnel-to-MCP-server hop lives. Second, Event ID 3 is high volume, so people filter it aggressively and lose the signal.

SYSMON — MINIMUM VIABLE ADDITIONS

```
<Sysmon schemaversion="4.90">
  <EventFiltering>

    <!-- 1: process creation. Catch tunnel agents and stdio bridges. -->
    <RuleGroup groupRelation="or">
      <ProcessCreate onmatch="include">
        <Image condition="end with">cloudflared.exe</Image>
        <Image condition="end with">ngrok.exe</Image>
        <Image condition="end with">devtunnel.exe</Image>
        <Image condition="end with">frpc.exe</Image>
        <CommandLine condition="contains any">tunnel --url;--authtoken;mcp-remote;supergateway;mcp-proxy</
        CommandLine>
        <CommandLine condition="contains">ssh -R</CommandLine>
      </ProcessCreate>
    </RuleGroup>

    <!-- 3: DO NOT blanket-exclude loopback. Include it for likely agent runtimes. -->
    <RuleGroup groupRelation="or">
      <NetworkConnect onmatch="include">
        <DestinationIp condition="is">127.0.0.1</DestinationIp>
        <DestinationIp condition="is">::1</DestinationIp>
      </NetworkConnect>
    </RuleGroup>

    <!-- 22: DNS. Cheapest high-signal source you have. -->
    <RuleGroup groupRelation="or">
      <DnsQuery onmatch="include">
        <QueryName condition="end with">argotunnel.com</QueryName>
        <QueryName condition="end with">trycloudflare.com</QueryName>
        <QueryName condition="end with">ngrok.com</QueryName>
        <QueryName condition="end with">ngrok.io</QueryName>
        <QueryName condition="end with">ngrok-free.app</QueryName>
        <QueryName condition="end with">devtunnels.ms</QueryName>
        <QueryName condition="end with">loca.lt</QueryName>
        <QueryName condition="end with">ts.net</QueryName>
        <QueryName condition="end with">equinox.io</QueryName>
      </DnsQuery>
    </RuleGroup>

    <!-- 11/23: file writes to MCP client configuration. -->
    <RuleGroup groupRelation="or">
      <FileCreate onmatch="include">
        <TargetFilename condition="end with">mcp.json</TargetFilename>
        <TargetFilename condition="end with">claude_desktop_config.json</TargetFilename>
        <TargetFilename condition="end with">mcp_config.json</TargetFilename>
      </FileCreate>
    </RuleGroup>
```

```
</EventFiltering>
</Sysmon>
```

Merge these rule groups into your existing configuration rather than replacing it. Test Event ID 3 loopback inclusion on a pilot group first — on a developer workstation it can be noisy, and you may need to scope it to specific parent processes.

6.4 Network layer

MCP classification at the gateway

As of 14 August 2026 Cloudflare Gateway classifies TLS-inspected requests as MCP by inspecting the `MCP-Protocol-Version` header, exposed as a boolean selector `experimental.is_mcp` in Gateway HTTP logs and policies. A Traffic Source selector distinguishes portal-proxied requests (`mcp_portal`) from direct connections, which is what makes T5 detectable. The baseline enforcement policy is:

CLOUDFLARE GATEWAY POLICY

```
experimental.is_mcp == true and not traffic.onramp in ("mcp_portal")
Action: Block
```

Even if you do not use Cloudflare, the technique generalises to any inspecting proxy: **classify on the header, not the hostname**. Write the equivalent rule for your own SWG.

STATE THE LIMITS WHEN YOU BRIEF THIS

Header-based classification requires TLS decryption. It does not see local stdio servers, off-network devices, Do-Not-Inspect categories, or traffic that never traverses the proxy. Presence of the header is a strong positive indicator; **absence proves nothing** — legacy clients, custom transports and non-conforming implementations may never send it. Any dashboard built on this is a floor on your MCP traffic, not a total.

What to do when you cannot decrypt

Degraded but not useless, in order of value: SNI from `ssl.log` or AWS Network Firewall gives you the destination hostname. JA3/JA4 client fingerprints identify the tunnel agent's TLS stack, which is distinct from a browser's. Connection duration and byte-ratio from `conn.log` or VPC Flow Logs identify long-lived low-volume sessions with an inverted download/upload ratio — characteristic of a tunnel serving requests rather than a client fetching pages. None of these tell you it is MCP; together they tell you it is a tunnel.

6.5 Identity layer

Three queries worth standing up as scheduled hunts:

- **New OAuth client registrations and consent grants** associated with AI tooling. DCR is deprecated as of 2026-07-28 in favour of client metadata documents, so in a modern estate a dynamic registration event is itself anomalous and worth alerting on.
- **Token audience mismatches** at the resource server. An `aud` claim that does not match the MCP server identifier is token passthrough, which the specification forbids outright. If your servers do not log this, that is the finding.
- **Service principals with broad scopes created by non-platform identities** — the low-code agent pattern where a maker's own credentials become the agent's effective identity.

6.6 Server layer

The NSA CSI is unambiguous here: log all tool and model invocations including the exact parameters, the identities involved, and where feasible cryptographic hashes of results, and integrate that telemetry into existing SIEM and detection pipelines. Most implementations do not. If you own the server, this is the highest-leverage thing you can build, because it is the only source that sees which tool ran with what intent.

Minimum fields for a record that is actually useful in an investigation:

Field	Why
<code>request_uuid</code> , <code>traceparent</code>	Correlation across client, gateway and server. Use the spec's W3C Trace Context key names.
<code>caller_identity</code> , <code>token_aud</code> , <code>client_id</code>	Who, verified. Not who the request claims to be.
<code>rpc_method</code> , <code>tool</code> , <code>risk_tier</code>	What was attempted and how much it matters. Risk-tiering at log time is what makes alerting tractable.
<code>args_digest</code> , <code>arg_keys</code>	Hash the arguments; log the key names in clear. Supports replay and duplicate detection without writing account numbers into a log store.
<code>peer_ip</code> , <code>x_forwarded_for</code> , provider trace header	Path taken. A provider trace header such as <code>Cf-Ray</code> present on an internal-only server means the request arrived through a tunnel.
<code>hdr_mcp_method</code> vs <code>rpc_method</code>	Log both and alert on disagreement. Under 2026-07-28 servers must reject mismatches; a mismatch means something rewrote one and not the other.
<code>outcome</code> , <code>reason</code> , <code>result_digest</code>	Did it run. Denials are as important as successes.

The lab server in Section 9 emits exactly this shape. Use it as a reference implementation when you write the requirement into your internal MCP server standard.

6.7 Blind spots you must write down

PUT THESE IN THE REPORT, NOT JUST THE APPENDIX

- **stdio is invisible to the network. Entirely.** No exceptions, no partial coverage.
- **No TLS inspection means no MCP classification.** Everything in Section 7's proxy content depends on decryption.
- **Off-network and BYOD devices are outside every network control** you own.
- **QUIC routes around HTTP proxies.** If UDP/443 and UDP/7844 are open outbound, your proxy is optional from the tunnel's point of view.
- **Encrypted Client Hello removes SNI** where deployed, taking your last no-decrypt signal with it.
- **Cached tool lists** mean a poisoned tool description may be in use for up to `ttlMs` after you fix the server.
- **Third-party MCP servers emit no audit you can read.** Your only record of what happened is your own client-side or gateway-side telemetry.

7. Detection content you can deploy

Tune thresholds and allowlist your known-good developer population before enabling any of this in production. On an engineering-heavy estate the tunnel rules will fire on legitimate work; that is expected, and the volume is itself a useful measurement of your exposure.

7.1 Sigma — tunnel agent execution

SIGMA

```

title: Reverse Tunnel Agent Publishing a Local MCP Server
id: 8f1c4a20-6c9e-4a1b-9f7d-2b5e0c3a7d41
status: experimental
description: |
  Detects execution of a reverse tunnel agent with arguments that point it at a
  local service. Combined with a co-occurring loopback listener this indicates a
  locally bound service has been made internet-reachable without any inbound
  firewall change. Covers taxonomy type T2.
references:
  - https://blog.cloudflare.com/mcp-security-updates/
  - https://attack.mitre.org/techniques/T1572/
author: AI Security Architecture
date: 2026/09/02
tags:
  - attack.command-and-control
  - attack.t1572
  - attack.exfiltration
  - attack.t1567
logsource:
  category: process_creation
detection:
  cloudflared:
    Image|endswith:
      - '\cloudflared.exe'
      - '/cloudflared'
    CommandLine|contains: 'tunnel'
  cloudflared_quick:
    CommandLine|contains|all:
      - 'tunnel'
      - '--url'
  ngrok:
    Image|endswith:
      - '\ngrok.exe'
      - '/ngrok'
    CommandLine|contains:
      - 'http '
      - 'tcp '
      - 'authtoken'
      - 'start '
  devtunnel:
    Image|endswith:
      - '\devtunnel.exe'
      - '/devtunnel'
    CommandLine|contains: 'host'
  ssh_reverse:
    CommandLine|re: '\bssh\b.*\s-R\s+\d{1,5}:'
  other_agents:
    Image|endswith:
      - '\frpc.exe'
      - '/frpc'
      - '/bore'

```

```

- '/pagekite'
CommandLine|contains:
- 'localhost'
- '127.0.0.1'
points_at_local:
CommandLine|contains:
- 'localhost:'
- '127.0.0.1:'
- 'http://localhost'
- 'http://127.0.0.1'
filter_known_good:
# replace with your approved build agents / dev VDI naming
Computer|startswith: 'BUILD-'
condition: >
( cloudflared or cloudflared_quick or ngrok or devtunnel
or ssh_reverse or (other_agents and points_at_local) )
and not filter_known_good
falsepositives:
- Legitimate developer workflow exposing a local service for AI assistant testing.
  This is the documented pattern for remote MCP development and will fire.
  Triage on: is the target service in scope, and is the host authorised?
level: high

```

7.2 Sigma — MCP client config tampering

SIGMA

```

title: MCP Client Configuration Created or Modified
id: c07b9e13-4d2a-4c88-8a35-1f9e77c4b6aa
status: experimental
description: |
  Detects writes to MCP client configuration files. A new server entry registers
  a tool source the model will trust and whose descriptions it will follow as
  instructions. Project-scoped .mcp.json files arrive with a git clone.
  Covers taxonomy types T1, T3 and MCP09 Shadow MCP Servers.
references:
- https://owasp.org/www-project-mcp-top-10/
author: AI Security Architecture
date: 2026/09/02
tags:
- attack.persistence
- attack.t1546
logsource:
  category: file_event
detection:
  config_files:
    TargetFilename|endswith:
      - '\mcp.json'
      - '/mcp.json'
      - '\.mcp.json'
      - '/.mcp.json'
      - '\claude_desktop_config.json'
      - '\mcp_config.json'
      - '/mcp_config.json'
      - '\.claude.json'
      - '/.claude.json'
  managed_policy:
    TargetFilename|contains:
      - 'managed-settings.json'
  writer_not_expected:
    Image|endswith:
      - '\powershell.exe'
      - '\cmd.exe'
      - '\wscript.exe'

```

```

- '\curl.exe'
- '/bash'
- '/curl'
- '/wget'
condition: (config_files and writer_not_expected) or managed_policy
falsepositives:
- Legitimate configuration management or a developer editing by script.
- Cloning a repository that contains a project-scoped .mcp.json.
level: medium

```

Writes to `managed-settings.json` fire unconditionally: that file is the enterprise policy control, and anything modifying it outside your configuration-management pipeline is a control-bypass attempt.

7.3 KQL — Defender / Sentinel

KQL — TUNNEL EXECUTION WITH LOOPBACK CORRELATION

```

// Tunnel agent started, correlated with a loopback listener on the same host
// within 10 minutes. This is the high-fidelity signal.
let window = 10m;
let tunnels =
    DeviceProcessEvents
    | where Timestamp > ago(7d)
    | where FileName in~ ("cloudflared.exe","ngrok.exe","devtunnel.exe","frpc.exe",
        "cloudflared","ngrok","devtunnel","frpc")
        or ProcessCommandLine has_any ("tunnel --url","--authtoken","ssh -R",
            "mcp-remote","supergateway","mcp-proxy")
    | project TunnelTime = Timestamp, DeviceId, DeviceName, AccountName,
        TunnelProc = FileName, TunnelCmd = ProcessCommandLine,
        TunnelPid = ProcessId, InitiatingProcessFileName;
let loopback =
    DeviceNetworkEvents
    | where Timestamp > ago(7d)
    | where RemoteIP in ("127.0.0.1", "::1")
    | where ActionType in ("ListeningConnectionCreated","ConnectionSuccess")
    | project LoopTime = Timestamp, DeviceId, LoopProc = InitiatingProcessFileName,
        LoopPort = RemotePort, LoopCmd = InitiatingProcessCommandLine;
tunnels
| join kind=inner loopback on DeviceId
| where abs(datetime_diff('second', LoopTime, TunnelTime)) <= toint(window / 1s)
| project TunnelTime, DeviceName, AccountName, TunnelProc, TunnelCmd,
    LoopProc, LoopPort, LoopCmd
| sort by TunnelTime desc

```

KQL — DNS TO TUNNEL PROVIDER INFRASTRUCTURE

```

DeviceNetworkEvents
| where Timestamp > ago(30d)
| where RemoteUrl has_any ("argotunnel.com","trycloudflare.com","ngrok.com",
    "ngrok.io","ngrok-free.app","devtunnels.ms",
    "loca.lt","lhr.life","serveo.net","pagekite.me",
    "equinox.io")
| summarize Lookups = count(),
    FirstSeen = min(Timestamp), LastSeen = max(Timestamp),
    Processes = make_set(InitiatingProcessFileName, 10),
    Domains = make_set(RemoteUrl, 15)
    by DeviceName, AccountName
| extend DurationDays = datetime_diff('day', LastSeen, FirstSeen)
// Persistent usage matters more than a one-off test
| where DurationDays >= 1 or Lookups > 20
| sort by Lookups desc

```

KQL — MCP CLIENT CONFIG WRITTEN, WITH THE NEW SERVER EXTRACTED

```

DeviceFileEvents
| where Timestamp > ago(30d)
| where FileName in~ ("mcp.json", ".mcp.json", "claude_desktop_config.json", "mcp_config.json")
| where ActionType in ("FileCreated", "FileModified")
| project Timestamp, DeviceName, AccountName, FolderPath, FileName,
    InitiatingProcessFileName, InitiatingProcessCommandLine, SHA256
| join kind=leftouter (
    DeviceNetworkEvents
    | where Timestamp > ago(30d)
    | where isnotempty(RemoteUrl)
    | project NetTime = Timestamp, DeviceName, RemoteUrl, NetProc = InitiatingProcessFileName
) on DeviceName
| where NetTime between (Timestamp .. (Timestamp + 5m))
| summarize DestinationsAfterConfigChange = make_set(RemoteUrl, 20)
    by Timestamp, DeviceName, AccountName, FolderPath, InitiatingProcessFileName

```

7.4 Splunk SPL

SPL — TUNNEL PROCESS EXECUTION

```

index=endpoint (sourcetype=Sysmon OR sourcetype=WinEventLog:Microsoft-Windows-Sysmon/Operational)
EventCode=1
| eval cmd=lower(CommandLine)
| where match(cmd, "cloudflared.*tunnel")
    OR match(cmd, "ngrok\s+(http|tcp|start)")
    OR match(cmd, "devtunnel\s+host")
    OR match(cmd, "ssh\s+.*-r\s+\d{1,5}:")
    OR match(cmd, "mcp-remote|supergateway|mcp-proxy|mcpo")
| eval tunnel_type=case(
    match(cmd, "cloudflared"), "T2-cloudflared",
    match(cmd, "ngrok"), "T2-ngrok",
    match(cmd, "devtunnel"), "T2-devtunnel",
    match(cmd, "ssh"), "T2-ssh-reverse",
    1, "T1-mcp-bridge")
| stats earliest(_time) as first_seen latest(_time) as last_seen
    values(User) as users values(ParentImage) as parents
    values(CommandLine) as cmdlines count
    by host tunnel_type
| convert ctime(first_seen) ctime(last_seen)
| sort - count

```

SPL — LONG-LIVED LOW-VOLUME OUTBOUND SESSIONS (TUNNEL SHAPE, FROM FLOW DATA)

```

index=network sourcetype=aws:cloudwatchlogs:vpcflow action=ACCEPT
| where (dstport=7844 OR dstport=443)
| stats sum(bytes) as total_bytes
    min(start) as flow_start max(end) as flow_end
    dc(dstaddr) as distinct_dsts count as flow_records
    by srcaddr dstaddr dstport
| eval duration_min = round((flow_end - flow_start)/60, 1)
| eval bytes_per_min = round(total_bytes / (duration_min + 1), 0)
| where duration_min > 60 AND bytes_per_min < 60000
| sort - duration_min
| head 100

```

The second search is deliberately loose. Its purpose is to build a baseline of long-lived outbound sessions per host, not to alert. Once you know what normal looks like on your estate, tighten it. Destination port 7844 alone is worth a standalone alert — very little legitimate traffic uses it.

7.5 Zeek and network analytics

ZEEL — SNI MATCHING TUNNEL PROVIDERS

```
# zeek-cut over ssl.log
cat ssl.log | zeek-cut -d ts id.orig_h id.resp_h server_name ja3 ja3s \
  | grep -Ei 'argotunnel|trycloudflare|ngrok|devtunnels\.ms|loca\.lt|ts\.net'

# Long-lived connections, tunnel-shaped: high duration, modest bytes,
# and more bytes returned to the originator than sent (serving, not fetching)
cat conn.log | zeek-cut -d ts id.orig_h id.resp_h id.resp_p proto duration \
  orig_bytes resp_bytes conn_state \
  | awk -F\t ' $6 > 3600 && $7 > 0 && $8 > 0 && ($8/$7) > 3 { print }'
```

ZEEL SCRIPT — FLAG MCP PROTOCOL HEADERS IN CLEARTEXT HTTP (LAB / INTERNAL SEGMENTS)

```
@load base/protocols/http

event http_header(c: connection, is_orig: bool, name: string, value: string) {
  if ( ! is_orig ) return;
  if ( name == "MCP-PROTOCOL-VERSION" )
    NOTICE([$note=Notice::Tally,
      $msg=fmt("MCP request to %s:%s version=%s",
        c$id$resp_h, c$id$resp_p, value),
      $conn=c]);
  if ( name == "MCP-NAME" )
    NOTICE([$note=Notice::Tally,
      $msg=fmt("MCP tool invoked: %s", value),
      $conn=c]);
}
```

Zeek uppercases header names in `http_header`. This only works on unencrypted HTTP — useful behind a TLS-terminating proxy, on an internal segment, or in the lab. It is the cleanest way to prove to a stakeholder that the tool name is on the wire.

7.6 Linux auditd and osquery

AUDITD RULES — /ETC/AUDIT/RULES.D/99-MCP-TUNNEL.RULES

```
# Execution of tunnel agents and MCP bridges.
# Path rules only fire if the binary is at that path; add -F exe= variants
# or use the execve name filter below for anything installed elsewhere.
-w /usr/bin/cloudflared -p x -k mcp_tunnel
-w /usr/local/bin/cloudflared -p x -k mcp_tunnel
-w /usr/bin/ngrok -p x -k mcp_tunnel
-w /usr/local/bin/ngrok -p x -k mcp_tunnel
-w /usr/bin/frpc -p x -k mcp_tunnel

# Any execve — post-filter on argv in your pipeline for tunnel/bridge keywords.
# High volume: scope to agent hosts, or drop in favour of eBPF.
-a always,exit -F arch=b64 -S execve -F auid>=1000 -F auid!=4294967295 -k proc_exec

# Writes to MCP client configuration.
-w /etc/claude-code/managed-settings.json -p wa -k mcp_config
-a always,exit -F arch=b64 -S openat -F dir=/home -F perm=wa -k home_write

# Reload: sudo augenrules --load && sudo systemctl restart auditd
# Query: sudo ausearch -k mcp_tunnel -i
# sudo ausearch -k proc_exec -i | grep -Ei 'cloudflared|ngrok|mcp-remote'
```

OSQUERY — SCHEDULED PACK

```
{
  "schedule": {
    "mcp_loopback_listeners": {
      "query": "SELECT p.pid, p.name, p.path, p.cmdline, l.address, l.port, u.username
        FROM listening_ports l
        JOIN processes p ON l.pid = p.pid
        LEFT JOIN users u ON p.uid = u.uid
        WHERE l.address IN ('127.0.0.1','::1') AND l.port != 0;",
      "interval": 300,
      "description": "Loopback listeners: candidate MCP servers invisible to network scans"
    },
    "mcp_tunnel_processes": {
      "query": "SELECT pid, name, path, cmdline, start_time, parent
        FROM processes
        WHERE name IN ('cloudflared','ngrok','frpc','bore','devtunnel','localtunnel')
          OR cmdline LIKE '%mcp-remote%'
          OR cmdline LIKE '%supergateway%'
          OR cmdline LIKE '%mcp-proxy%'
          OR cmdline LIKE '%tunnel --url%'
          OR cmdline LIKE '%ssh%-R%';",
      "interval": 300,
      "description": "Reverse tunnel agents and stdio-to-HTTP MCP bridges"
    },
    "mcp_client_configs": {
      "query": "SELECT path, size, mtime, sha256
        FROM file JOIN hash USING (path)
        WHERE path LIKE '/home/%/.cursor/mcp.json'
          OR path LIKE '/home/%/.claude.json'
          OR path LIKE '/home/%/.claude/%.json'
          OR path LIKE '/home/%/.config/Code/User/mcp.json';",
      "interval": 3600,
      "description": "Declared MCP server inventory, with hashes for drift detection"
    },
    "mcp_outbound_to_tunnels": {
      "query": "SELECT s.pid, p.name, p.cmdline, s.remote_address, s.remote_port
        FROM process_open_sockets s JOIN processes p ON s.pid = p.pid
        WHERE s.remote_port IN (7844, 443)
          AND p.name IN ('cloudflared','ngrok','tailscaled','frpc','devtunnel');",
      "interval": 300
    }
  }
}
```

7.7 The high-fidelity correlation

Everything above produces individually noisy signals. This is the one that is worth paging on.

THE RULE

A process holds a loopback LISTEN socket, and within the same window a known reverse tunnel agent holds an outbound ESTABLISHED session, on the same host.

Neither condition alone is remarkable — developers run local servers constantly, and tunnel agents have legitimate uses. Together they mean a locally bound service is very likely reachable from the public internet, and no inbound firewall rule changed to make that true. Add a third condition — the loopback listener's command line matches an MCP server pattern — and precision goes up sharply.

The `mcp_hunt.sh` and `Find-McpTunnel.ps1` scripts shipped alongside this guide implement exactly this correlation for point-in-time triage. For continuous detection, implement it in your SIEM using the KQL in 7.3 as the pattern.

7.8 Tool-description drift detection

Not a tunnel detection, but it belongs in the same programme because it closes the other half of the shadow-MCP risk. Tool descriptions are functionally instructions: the model treats a tool's metadata with the same authority as its system prompt. A server that presents benign descriptions at approval time and changes them later — the rug-pull pattern — alters agent behaviour with no code change and, in most deployments, no re-approval step.

The mitigation is trust-on-first-use pinning: hash every tool description and its metadata when you approve a server, recheck on every reconnect, and diff.

BASH — PIN AND DIFF A TOOL CATALOGUE

```
#!/usr/bin/env bash
# mcp_pin.sh
# Fetch tools/list, canonicalise, hash each tool, diff against baseline.
set -euo pipefail
URL="$1"; BASE="${2:-tools.baseline}"

curl -sS "$URL" \
  -H 'Content-Type: application/json' \
  -H 'MCP-Protocol-Version: 2026-07-28' \
  -H 'Mcp-Method: tools/list' \
  -d '{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{}}' \
| jq -r '.result.tools[]' \
  | [.name,
     (({description, inputSchema, title} | tostring) | @base64)] \
  | @tsv \
| while IFS=$'\t' read -r name blob; do
  printf '%s %s\n' "$name" "$((printf '%s' "$blob" | sha256sum | cut -c1-32))"
done | sort > tools.current

if [ ! -f "$BASE" ]; then
  cp tools.current "$BASE"
  echo "baseline written to $BASE"; exit 0
fi

if diff -u "$BASE" tools.current; then
  echo "OK: tool catalogue unchanged"
else
  echo "ALERT: tool description or schema drift detected on $URL" >&2
  exit 1
fi
```

Run it on a schedule shorter than the server's advertised `ttlMs`, and on every reconnect. Alert on any diff, then require a human re-approval before the server is used again. Also scan descriptions for invisible Unicode — tag characters and zero-width spaces smuggle instructions past human review while still reaching the model.

8. Prevention: what actually holds

8.1 Control tiers

Tier	Control	Stops	Cost and caveat
1 Prevents	Application allowlisting (WDAC / AppLocker / fapolicyd)	T1 T2	High deployment effort, highest durability. The only control that addresses root cause.
	Default-deny egress through a TLS-inspecting proxy, with UDP blocked	T1 T2 T3	Architectural change. Breaks things before it protects things. Non-negotiable for agent-hosting subnets.
	Server-side tool authorization with risk tiers	T3 T4 T5	Requires you to own the server. Last line before execution, and the only one an end user cannot bypass by switching clients.
	Enterprise-managed client policy (MCP server allowlist)	T1 T3	Only covers managed clients on managed devices. Deploy anyway.
2 Raises cost	DNS policy blocking tunnel provider zones	T2	Hours to deploy. Evaded by an IP literal or DoH. Excellent detection value regardless.
	Package registry proxy: block or pin bridge packages	T1	Low effort if you already run an internal registry.
	OAuth hardening: pre-registered clients, audience validation, no passthrough	T3 T5	IAM work. Aligns with what 2026-07-28 already pushes you toward.
3 Detects only	Gateway MCP classification and dashboards	—	Requires TLS inspection. Start here for visibility before you enforce anything.
	Periodic network scanning for exposed MCP servers	—	NSA-recommended. Misses every loopback-bound server behind a tunnel — say so when you report coverage.

8.2 Tier 1 — binary and execution control

WINDOWS — APPLOCKER PUBLISHER/PATH DENY (TEST IN AUDIT MODE FIRST)

```
# Enumerate what is already present before you write policy
Get-ChildItem -Path C:\ -Include cloudflared.exe,ngrok.exe,devtunnel.exe,frpc.exe `
  -Recurse -ErrorAction SilentlyContinue |
  Select-Object FullName, Length, LastWriteTime,
    @{n='SHA256';e={(Get-FileHash $_.FullName).Hash}}
```

```
# AppLocker deny rule fragment – merge into your existing policy XML
# <FilePathRule Id="a1b2c3d4-0000-0000-0000-000000000001"
#     Name="Deny reverse tunnel agents"
#     Description="MCP tunnel control - see AI Security standard"
#     UserOrGroupSid="S-1-1-0" Action="Deny">
#   <Conditions>
#     <FilePathCondition Path="*\cloudflared.exe" />
#     <FilePathCondition Path="*\ngrok.exe" />
#     <FilePathCondition Path="*\frpc.exe" />
#     <FilePathCondition Path="*\devtunnel.exe" />
#   </Conditions>
# </FilePathRule>
```



```
# Verify enforcement mode is what you think it is
Get-AppLockerPolicy -Effective -Xml | Select-String 'EnforcementMode'
```

PATH RULES ARE A SPEED BUMP, NOT A WALL

A renamed binary defeats a path rule in seconds. Use publisher rules where the binary is signed, and hash rules where it is not. WDAC with a managed allowlist is strictly better than AppLocker deny-listing, because deny-listing an ever-growing set of tunnel tools is a losing race — there are dozens and the list grows monthly. Allowlist what is permitted to execute and the problem inverts in your favour.

LINUX — FAPOLICYD DENY (RHEL / AMAZON LINUX)

```
# /etc/fapolicyd/rules.d/25-mcp-tunnel.rules (before the default allow rules)
deny_audit perm=execute all : path=/usr/bin/cloudflared
deny_audit perm=execute all : path=/usr/local/bin/cloudflared
deny_audit perm=execute all : path=/usr/bin/ngrok
deny_audit perm=execute all : path=/usr/local/bin/ngrok
deny_audit perm=execute all : path=/usr/bin/frpc

sudo fagenrules --load
sudo systemctl restart fapolicyd
sudo journalctl -u fapolicyd -f      # watch denials

# Stronger: allowlist model — permit execution only from trusted, RPM-owned paths
# and deny everything under user-writable directories.
deny_audit perm=execute all : dir=/home/
deny_audit perm=execute all : dir=/tmp/
deny_audit perm=execute all : dir=/var/tmp/
```

The three `dir=` denies at the bottom are the high-value lines. Almost every tunnel binary arrives as a user-downloaded file in a home directory. Denying execution from user-writable paths stops the entire class, including the ones you have not heard of yet — and it will break some legitimate developer workflows, so pilot it.

8.3 Tier 1 — egress architecture

Target state for any subnet that hosts an agent, an MCP server, or a copilot runtime:

1. **No direct internet route.** No public IP, no default route to an internet gateway. All egress through an explicit forward proxy.
2. **Proxy enforces an allowlist** of destination hostnames, not IPs, and performs TLS inspection so MCP headers are visible.
3. **UDP/443 and UDP/7844 blocked outbound.** Without this, QUIC bypasses your HTTP proxy entirely and the whole design is decorative.
4. **Provider zones blocked at the resolver** and every attempt logged.
5. **DNS-over-HTTPS blocked** to anything other than your own resolver, or clients will resolve tunnel hostnames around your DNS policy.

SQUID — MINIMAL ALLOWLIST FOR AN AGENT SUBNET

```
# /etc/squid/squid.conf
acl agent_net src 10.20.30.0/24

acl approved_dst dstdomain "/etc/squid/approved-mcp-destinations.txt"
acl tunnel_dst dstdomain .argotunnel.com .trycloudflare.com .ngrok.com \
    .ngrok.io .ngrok-free.app .devtunnels.ms \
    .localt .lhr.life .serveo.net .pagekite.me \
    .ts.net .equinox.io
```

```
http access deny agent_net tunnel dst
http_access allow agent_net approved_dst
http_access deny agent_net all

# Log the full URL so MCP endpoint paths are visible in access.log
logformat mcpfmt %ts.%03tu %6tr %>a %Ss/%03>Hs %<st %rm %ru %[un %Sh/%<a %mt
access_log /var/log/squid/access.log mcpfmt
```

The NSA CSI names a filtering outbound proxy (it cites Squid and tinyproxy by name) or an enterprise DLP solution, scoped to specific resource URLs and access methods, as the recommended way to protect an MCP environment from unintended leakage. Quote that when someone asks why the agent subnet needs a proxy.

LINUX HOST-LEVEL EGRESS DENY AS DEFENCE IN DEPTH

```
# Block QUIC outbound so nothing routes around the proxy
sudo iptables -A OUTPUT -p udp --dport 443 -j REJECT
sudo iptables -A OUTPUT -p udp --dport 7844 -j REJECT
sudo iptables -A OUTPUT -p tcp --dport 7844 -j REJECT

# Deny direct 443 for the account the agent runs as; force it through the proxy
sudo iptables -A OUTPUT -p tcp --dport 443 -m owner --uid-owner agentsvc \
    -d 10.20.30.10 -j ACCEPT # the proxy itself
sudo iptables -A OUTPUT -p tcp --dport 443 -m owner --uid-owner agentsvc -j REJECT

sudo iptables-save | sudo tee /etc/sysconfig/iptables # or netfilter-persistent save
```

8.4 Tier 1 — the governed MCP path

The pattern that works is a single sanctioned endpoint through which all MCP traffic flows: an MCP gateway or portal that presents one managed endpoint to clients, enforces identity, exposes a curated tool catalogue, applies DLP, and produces tool-level audit. Approved servers move behind it; everything else is blocked.

Three things make it real rather than aspirational:

1. **Discovery first, enforcement second.** Turn on classification and dashboards, compare observed destinations against your approved list, and move approved servers behind the portal. Enforce only once the sanctioned path covers what people actually need — a portal that cannot reach the servers your teams require guarantees bypass.
2. **Block the direct path at the network** once coverage is adequate. The Cloudflare formulation — block anything classified as MCP whose on-ramp is not the portal — is the right shape whatever product you use.
3. **Reject direct connections at the origin.** Network policy only covers managed devices on managed paths. Your own MCP servers must independently refuse requests that did not arrive via the portal: an Access policy, a source-IP restriction, or a signed header the portal adds and the origin verifies. Without this, T5 is unaddressed for anyone off the corporate network.

SERVER-SIDE ORIGIN CHECK — REJECT NON-PORTAL TRAFFIC

```
# Add to the request handler, before any tool dispatch.
PORTAL_SHARED_SECRET = os.environ["MCP_PORTAL_HMAC_KEY"]

def arrived_via_portal(headers) -> bool:
    sig = headers.get("X-Portal-Signature")
    ts = headers.get("X-Portal-Timestamp")
    if not sig or not ts:
        return False
    if abs(time.time() - int(ts)) > 60: # replay window
```

```

        return False
    expected = hmac.new(PORTAL_SHARED_SECRET.encode(),
                        ts.encode(), hashlib.sha256).hexdigest()
    return hmac.compare_digest(sig, expected)

if not arrived_via_portal(self.headers):
    audit({"event": "mcp.request", "outcome": "reject",
          "reason": "portal_bypass", "peer_ip": peer})
    self._rpc_error(req_id, -32001, "Direct access denied; use the MCP portal")
    return

```

8.5 Tier 2 — identity and authorization

The 2026-07-28 authorization changes push you toward the pattern your IAM team already wanted. Implement all of it:

- **No Dynamic Client Registration.** Deprecated in favour of client metadata documents. Pre-register OAuth clients with fixed client IDs, secrets, callback URLs and scopes. In a modern estate a DCR event should alert.
- **Validate the `iss` parameter** per RFC 9207 before redeeming an authorization code. This closes the authorization-server mix-up hole.
- **Validate token audience on every request.** Reject any token whose `aud` does not match the MCP server identifier, even if it is signed and unexpired. This is the single check that stops token passthrough.
- **Never forward a caller token downstream.** Use token exchange when a downstream service needs to act on behalf of the user. The specification forbids passthrough outright.
- **Per-client consent registry** for any proxying server: maintain approved `client_id` values per user, check before initiating a third-party flow, and never set the consent cookie before the user has actually approved.
- **Bind each request to a specific client identity, approved scope and approved operation** server-side. Server-level access control alone leaves the confused-deputy problem wide open.

8.6 Tier 2 — money-movement specific controls

THE RULE TO WRITE INTO STANDARD

No autonomous write path to money movement. An agent may prepare, validate, enrich and summarise a payment instruction. It may not complete one. The commit step requires a human action taken against the system of record, on a channel the agent does not control, showing data the agent did not render.

Control	Implementation
Risk-tier every tool	Tag each tool <code>read</code> , <code>read-sensitive</code> , <code>write</code> , <code>write-critical</code> in its definition and enforce at the handler. Reads pass through; writes get attribution and an audit event; critical actions are blocked or require approval before the handler runs. Because the check lives at the server, a user cannot bypass it by switching clients or disabling a local hook.
Out-of-band confirmation	Beneficiary details displayed for confirmation must be re-read from the system of record, not echoed from the agent's context. If the model was manipulated, the summary it renders is manipulated too.

Control	Implementation
Preserve dual control at the core	Maker-checker must be enforced in the core banking platform. If the second approval is a step in the agent's workflow definition, it is one compromise away from being a formality.
Idempotency keys	Mandatory on every state-changing tool. MCP does not enforce idempotency and the NSA CSI flags this explicitly. Without it, a retry is a second payment.
Velocity and anomaly limits at the tool	Rate-limit <code>tools/call</code> per identity per tool. An agent can invoke a tool thousands of times before a human notices; a limit that would never inconvenience a human is a hard stop for a runaway agent.
Segregate the context	An agent with a money-movement tool must not also hold a tool that ingests untrusted external content. Enforce at agent definition review, not at runtime.
Deterministic orchestration	For money-movement agents, use fixed topics rather than letting the model choose tools. Removes tool selection as an injection target.

8.7 Cloud and EC2 specifics

AWS — CONTROLS THAT ACTUALLY APPLY

```
# 1. Confirm the security group has no inbound rules that matter.
# (This proves the point in Section 5 – and changes nothing about the tunnel.)
aws ec2 describe-security-groups --group-ids sg-0abc123 \
  --query 'SecurityGroups[0].IpPermissions'

# 2. Egress restriction is the control that DOES apply. Remove allow-all.
aws ec2 revoke-security-group-egress --group-id sg-0abc123 \
  --protocol -1 --port -1 --cidr 0.0.0.0/0

aws ec2 authorize-security-group-egress --group-id sg-0abc123 \
  --ip-permissions 'IpProtocol=tcp,FromPort=443,ToPort=443,
  IpRanges=[{CidrIp=10.20.30.10/32,Description="forward proxy only"}]'

# 3. Route 53 Resolver DNS query logging – cheap, high signal.
aws route53resolver create-resolver-query-log-config \
  --name mcp-tunnel-dns --destination-arn arn:aws:logs:us-east-1:ACCT:log-group:/dns/queries

aws route53resolver associate-resolver-query-log-config \
  --resolver-query-log-config-id rqlc-0abc --resource-id vpc-0abc123

# 4. Route 53 Resolver DNS Firewall – block the provider zones outright.
aws route53resolver create-firewall-domain-list --name tunnel-providers
aws route53resolver update-firewall-domains --firewall-domain-list-id rslvr-fdl-0abc \
  --operation ADD --domains "*.argotunnel.com" "*.trycloudflare.com" \
  "*.ngrok.com" "*.ngrok.io" "*.ngrok-free.app" \
  "*.devtunnels.ms" "*.loca.lt" "*.ts.net"

# 5. VPC endpoints so legitimate AWS API traffic never needs internet egress.
aws ec2 create-vpc-endpoint --vpc-id vpc-0abc123 \
  --service-name com.amazonaws.us-east-1.s3 --vpc-endpoint-type Gateway

# 6. VPC Flow Logs – necessary, insufficient. No hostname, no SNI, no headers.
aws ec2 create-flow-logs --resource-type VPC --resource-ids vpc-0abc123 \
  --traffic-type ALL --log-destination-type cloud-watch-logs \
  --log-group-name /vpc/flowlogs --deliver-logs-permission-arn arn:aws:iam::ACCT:role/flowlogs
```

THE ONE AWS CONTROL MOST TEAMS SKIP

AWS Network Firewall with TLS SNI-based domain rules. VPC Flow Logs cannot tell you where a connection went by name; Network Firewall can, and can block on it. For any subnet hosting agents or MCP servers, a Network Firewall with a domain allowlist is the difference between "we saw an outbound flow to a Cloudflare IP" and "we blocked an attempt to reach trycloudflare.com and know which instance made it."

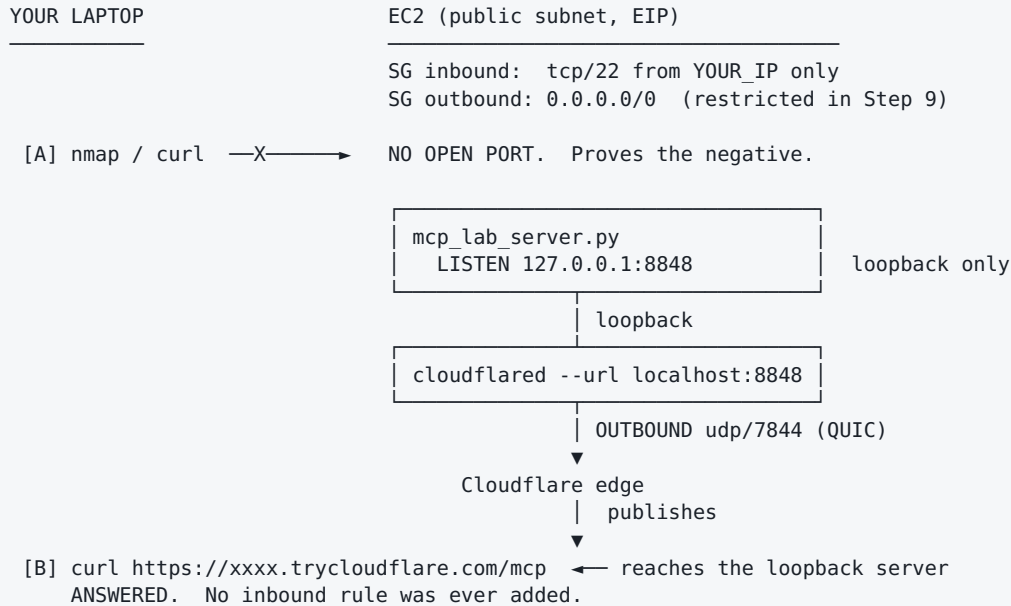
8.8 Copilot Studio and low-code specifics

- **Block the generic HTTP connector** via DLP connector policy. It is an arbitrary egress channel and on some platforms it bypasses tenant isolation by design.
- **Restrict outbound email actions** from agents, or constrain recipients to a hard-coded internal address. Email is the exfiltration channel in the EchoLeak class of attack.
- **Require approval flows as human-in-the-loop** for any action that sends data externally or changes a record of consequence.
- **Inventory agents and their MCP servers continuously.** Use the platform's AI agent inventory if you have it; script it against the platform API if you do not.
- **Wire the injection-shield telemetry into the SIEM.** Verify events actually arrive; the integration is frequently not on by default.
- **Review agent definitions as code.** Topics, tools and knowledge sources together determine the security posture. That triple should go through review before publication, with the three trifecta questions from Section 2.1 answered in writing.

9. The lab: reproduce it end to end

9.0 Topology and build

TOPOLOGY



BUILD THE INSTANCE

```
# Amazon Linux 2023
sudo dnf install -y python3 tcpdump iproute audit bind-utils jq nmap-ncat git

# Ubuntu 24.04
sudo apt-get update && sudo apt-get install -y \
    python3 tcpdump iproute2 auditd dnsutils jq netcat-openbsd git

mkdir -p ~/mcp-lab && cd ~/mcp-lab
# copy mcp_lab_server.py, mcp_probe.py and mcp_hunt.sh into this directory
chmod +x mcp_hunt.sh

# cloudflared (Quick Tunnels need no account)
curl -fsSL -o cloudflared \
    https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64
chmod +x cloudflared && sudo mv cloudflared /usr/local/bin/
cloudflared --version
```

Step 1 Baseline and prove the negative

Record the starting state. When you brief this later, the value is in the before/after contrast, so capture it properly.

ON THE EC2 INSTANCE

```
INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $(curl -sX PUT \
    http://169.254.169.254/latest/api/token \
    -H 'X-aws-ec2-metadata-token-ttl-seconds: 300')\" \
    http://169.254.169.254/latest/meta-data/instance-id)
```

```
echo "instance: $INSTANCE_ID"

# Every listening socket on the box, before we change anything
ss -tulnp | tee ~/mcp-lab/baseline-listeners.txt

# Host firewall state
sudo iptables -L -n -v | tee ~/mcp-lab/baseline-iptables.txt
```

FROM YOUR LAPTOP — THE SECURITY GROUP AND THE SCAN

```
aws ec2 describe-instances --instance-ids i-0abc123 \
  --query 'Reservations[].Instances[].SecurityGroups' --output table

aws ec2 describe-security-groups --group-ids sg-0abc123 \
  --query 'SecurityGroups[0].{Inbound:IpPermissions,Outbound:IpPermissionsEgress}'

EIP=1.2.3.4
nmap -Pn -p- --max-retries 1 -T4 $EIP | tee scan-before.txt
curl -m 5 -v http://$EIP:8848/mcp      # expect: connection timed out
```

EXPECTED RESULT

`nmap` reports only `tcp/22` open (from your source address). Nothing on 8848. `curl` times out. Inbound exposure is zero and you have the artifact to prove it.

Step 2 Stand up an MCP server on loopback

ON EC2

```
cd ~/mcp-lab
nohup python3 mcp_lab_server.py --port 8848 --audit ~/mcp-lab/mcp-audit.jsonl \
  > ~/mcp-lab/server.out 2>&1 &
sleep 1

# Confirm the bind address. This is the whole point: 127.0.0.1, not 0.0.0.0.
ss -tlnp | grep 8848
```

EXPECTED OUTPUT

```
LISTEN 0  5  127.0.0.1:8848  0.0.0.0:*  users:(("python3",pid=3412,fd=3))
```

EXERCISE IT LOCALLY

```
python3 mcp_probe.py http://127.0.0.1:8848/mcp discover
python3 mcp_probe.py http://127.0.0.1:8848/mcp list
python3 mcp_probe.py http://127.0.0.1:8848/mcp call get_balance account_id=ACCT-1001
python3 mcp_probe.py http://127.0.0.1:8848/mcp call add_beneficiary \
  account_id=ACCT-1001 beneficiary_name=NORTHWIND account_number=0044179912

# The write tool produces an audit record with risk_tier=write-critical
jq 'select(.risk_tier=="write-critical")' ~/mcp-lab/mcp-audit.jsonl
```

SAMPLE AUDIT RECORD — THIS IS THE SHAPE TO REQUIRE OF EVERY INTERNAL MCP SERVER

```
{
  "arg_keys": ["account_id", "account_number", "beneficiary_name"],
  "args_digest": "sha256:7651c4569a3e038e36c31edb56d8c5ab",
  "auth_present": false,
  "event": "mcp.request",
```

```

"hdr_mcp_method": "tools/call",
"hdr_mcp_name": "add_beneficiary",
"hdr_protocol_version": "2026-07-28",
"outcome": "ok",
"peer_ip": "127.0.0.1",
"request_uuid": "327805e5-208d-4a72-92a1-5249a7471aa9",
"risk_tier": "write-critical",
"rpc_method": "tools/call",
"tool": "add_beneficiary",
"ts": "2026-09-02T14:22:31.004+00:00"
}

```

PROVE THE HEADER-VS-BODY ENFORCEMENT (2026-07-28 REQUIREMENT, AND A TAMPER SIGNAL)

```

python3 mcp_probe.py http://127.0.0.1:8848/mcp legacy # omits routing headers
# → HTTP 400 "MCP-Protocol-Version header required"
jq 'select(.outcome=="reject")' ~/mcp-lab/mcp-audit.jsonl

```

Step 3 Confirm it is unreachable

FROM YOUR LAPTOP

```

curl -m 5 -v http://$EIP:8848/mcp
nmap -Pn -p 8848 $EIP
python3 mcp_probe.py http://$EIP:8848/mcp list

```

All three fail. The service is running, answering requests, and is completely unreachable from the internet. Nothing about the network configuration is going to change in the next step.

Step 4 Arm telemetry before you tunnel

Start collection *first*. The most instructive part of this lab is comparing what each source captured against what actually happened.

TERMINAL 1 — PACKET CAPTURE

```

# Everything except your own SSH session
sudo tcpdump -i any -nn -s 0 -w ~/mcp-lab/tunnel.pcap \
  "not (port 22 and host $(echo $SSH_CLIENT | awk '{print $1}'))"

```

TERMINAL 2 — SOCKET STATE, ONCE A SECOND

```

while true; do
  printf '=== %s\n' "$(date -u +%H:%M:%S)"
  ss -tanp state established '( sport = :7844 or dport = :7844 or dport = :443 )' 2>/dev/null
  ss -tlnp | grep -E '8848|LISTEN.*127\.0\.0\.1'
  sleep 1
done | tee ~/mcp-lab/socket-timeline.txt

```

TERMINAL 3 — AUDIT RULES AND PROCESS/DNS WATCH

```

sudo tee /etc/audit/rules.d/99-mcp-tunnel.rules >/dev/null <<'EOF'
-w /usr/local/bin/cloudflared -p x -k mcp_tunnel
-a always,exit -F arch=b64 -S execve -F auid=>1000 -F auid!=4294967295 -k proc_exec
EOF
sudo augenrules --load && sudo systemctl restart auditd

```



```
# DNS: watch the resolution of provider infrastructure in real time
sudo tcpdump -i any -nn -l 'udp port 53' | tee ~/mcp-lab/dns.txt
```

Step 5 Open the tunnel

TERMINAL 4 — ONE COMMAND, NO ACCOUNT, NO FIREWALL CHANGE

```
cloudflared tunnel --url http://localhost:8848 2>&1 | tee ~/mcp-lab/cloudflared.log
```

EXPECTED OUTPUT

```
INF Thank you for trying Cloudflare Tunnel. Doing so, without a Cloudflare account,
  is a quick way to experiment and try it out.
INF +-----+
INF | Your quick Tunnel has been created! Visit it at: |
INF | https://spectacular-marine-widely-notion.trycloudflare.com |
INF +-----+
INF Registered tunnel connection connIndex=0 protocol=quic
INF Registered tunnel connection connIndex=1 protocol=quic
```

CAPTURE THE URL AND INSPECT THE NEW SOCKET STATE

```
TUNNEL_URL=$(grep -oE 'https://[a-z0-9-]+\.\trycloudflare\.com' \
  ~/mcp-lab/cloudflared.log | head -1)
echo "$TUNNEL_URL" | tee ~/mcp-lab/tunnel-url.txt

# Look closely: OUTBOUND only. No new listener on a routable address.
ss -tanp | grep -i cloudflared
ss -uanp | grep -i cloudflared
sudo ss -tulnp | grep -v 127.0.0.1
```

Step 6 Prove the inversion

FROM YOUR LAPTOP — THE DEMONSTRATION

```
TUNNEL_URL=https://spectacular-marine-widely-notion.trycloudflare.com

# 1. Direct to the EIP: still nothing. Nothing changed at the network layer.
nmap -Pn -p 8848 $EIP
curl -m 5 http://$EIP:8848/mcp

# 2. Via the tunnel hostname: the loopback-bound server answers.
curl -sS "$TUNNEL_URL/mcp" \
  -H 'Content-Type: application/json' \
  -H 'MCP-Protocol-Version: 2026-07-28' \
  -H 'Mcp-Method: tools/list' \
  -d '{"jsonrpc": "2.0", "id": 1, "method": "tools/list", "params": {}}' | jq .

# 3. Invoke the write-critical tool from the public internet.
python3 mcp_probe.py "$TUNNEL_URL/mcp" call add_beneficiary \
  account_id=ACCT-1001 beneficiary_name=EXTERNAL-CALLER account_number=1234509876
```

THIS IS THE FINDING

An anonymous request from the public internet just invoked a write-tier tool on a service bound to `127.0.0.1`, on a host whose security group has **no inbound rule permitting it**, and no configuration in your cloud account, your firewall or your network changed to allow it. The security group is still correct. The scan is still clean. Screenshot the two `nmap` outputs side by side — that pairing is the most persuasive slide you will produce from this lab.

ON EC2 — SEE IT ARRIVE FROM OUTSIDE

```
jq 'select(.peer_ip != "127.0.0.1") // select(.cf_ray != null)' ~/mcp-lab/mcp-audit.jsonl
tail -5 ~/mcp-lab/mcp-audit.jsonl | jq '{ts, tool, risk_tier, peer_ip, x_forwarded_for, cf_ray, user_agent}'
```

NOTE THE HEADER ARTIFACT

Requests arriving through the tunnel carry `X-Forwarded-For` and a `Cf-Ray` trace header. On a server that is supposed to be internal-only, **the presence of a CDN trace header is a high-confidence indicator that the request arrived through a tunnel**. That is a one-line detection you can add to any internal MCP server today: alert whenever a request carries a provider trace header, and log the value.

Step 7 Read the telemetry, find the gap**PROCESS EXECUTION**

```
sudo ausearch -k mcp_tunnel -i | tail -20
sudo ausearch -k proc_exec -i | grep -i cloudflared | tail -5
ps -eo pid,ppid,user,etimes,args | grep -i [c]loudflared
```

DNS

```
grep -Ei 'argotunnel|trycloudflare|cloudflare' ~/mcp-lab/dns.txt | head -20
```

FLOWS — NOTE WHAT IS MISSING

```
# Outbound QUIC to the Cloudflare edge on 7844
sudo tcpdump -nn -r ~/mcp-lab/tunnel.pcap 'udp port 7844' | head -20

# Direction check: everything is outbound-initiated
sudo tcpdump -nn -r ~/mcp-lab/tunnel.pcap 'tcp[tcpflags] & tcp-syn != 0 and tcp[tcpflags] & tcp-ack == 0' | head

# The tunnel-to-server hop, on loopback, invisible to any network sensor
sudo tcpdump -nn -r ~/mcp-lab/tunnel.pcap 'host 127.0.0.1 and port 8848' | head
```

RUN THE HOST TRIAGE SCRIPT

```
sudo bash ~/mcp-lab/mcp_hunt.sh
sudo bash ~/mcp-lab/mcp_hunt.sh --json | jq -c 'select(.category=="CORRELATION_HIT")'
```

Source	Captured	Missed
auditd / process	<code>cloudflared</code> execution with full argv including <code>--url http://localhost:8848</code> , parent, uid, timestamp.	That a tool was called. Which tool. By whom.

Source	Captured	Missed
DNS	Resolution of *.argotunnel.com / api.trycloudflare.com . High signal, low volume.	Everything after the lookup.
pcap / flow	Outbound UDP/7844 to Cloudflare. Long-lived. Byte counts.	No SNI (QUIC), no hostname, no HTTP, no MCP, no tool name. Opaque.
Loopback pcap	Nothing useful without a host sensor — and Sysmon excludes loopback by default.	The entire request/response.
Security group / VPC Flow Logs	An outbound ACCEPT flow to a Cloudflare address.	That anything was published. This is the control everyone assumes is protecting them.
MCP server audit	Tool name, risk tier, argument digest, external peer, XFF, Cf-Ray, outcome.	Nothing that matters — <i>but only because you own this server.</i>

THE LESSON TO CARRY BACK

Network telemetry tells you a tunnel exists. Only the server-side audit tells you what was done through it. For third-party MCP servers you do not own, that record does not exist and never will — which is why the governed-path architecture in Section 8.4 matters: the portal is where you manufacture the audit trail that the third party will not give you.

Step 8 Add inspection, recover the signal

Now demonstrate what a TLS-inspecting egress proxy buys you. This is the argument for the control in 8.3, made with evidence.

ON EC2 — MITMPROXY AS A STAND-IN FOR YOUR SWG

```
pip install --user mitmproxy
mitmdump --mode regular --listen-port 8080 \
  --set flow_detail=3 -w ~/mcp-lab/mcp-flows.mitm &

# Force the probe through it, trusting the interception CA
export HTTPS_PROXY=http://127.0.0.1:8080
export SSL_CERT_FILE=~/.mitmproxy/mitmproxy-ca-cert.pem
python3 mcp_probe.py "$TUNNEL_URL/mcp" call get_balance account_id=ACCT-1001
```

EXTRACT EXACTLY WHAT A GATEWAY POLICY WOULD KEY ON

```
mitmdump -nr ~/mcp-lab/mcp-flows.mitm \
  -s /dev/stdin <<'PY'
def response(flow):
    h = flow.request.headers
    if "MCP-Protocol-Version" in h:
        print("MCP host=%-38s path=%-12s method=%-12s tool=%s status=%s" % (
            flow.request.host, flow.request.path,
            h.get("Mcp-Method", "-"), h.get("Mcp-Name", "-"),
            flow.response.status_code))
PY
```

EXPECTED OUTPUT

```
MCP host=spectacular-marine-widely-notion.trycloudflare.com path=/mcp
method=tools/call tool=get_balance status=200
```

WHAT YOU JUST PROVED

With decryption you recover the destination, the endpoint path, the operation and **the tool name** — without parsing a single JSON body. That is precisely the signal a modern gateway keys on, and it is what makes a policy like "block `tools/call` to any destination that is not the approved portal" implementable at the network layer. Without decryption you have an opaque QUIC flow. This single comparison is the strongest evidence you can present for TLS inspection on agent-hosting subnets.

Step 9 Test the preventive controls

Apply each control in turn, confirm it breaks the tunnel, then revert. Run `curl "$TUNNEL_URL/mcp"` from your laptop after each to verify.

9A — DNS BLOCKING

```
sudo cp /etc/hosts /etc/hosts.bak
printf '0.0.0.0 api.trycloudflare.com\n0.0.0.0 update.argotunnel.com\n' | sudo tee -a /etc/hosts
sudo pkill cloudflared
cloudflared tunnel --url http://localhost:8848      # fails to register
sudo cp /etc/hosts.bak /etc/hosts
```

Result: a new tunnel cannot start. An *already-established* tunnel keeps running — DNS blocking prevents new tunnels, it does not tear down live ones. Note that distinction in your control write-up; it changes your incident response steps.

9B — EGRESS BLOCKING (THE CONTROL THAT ACTUALLY WORKS AT THE NETWORK LAYER)

```
sudo iptables -A OUTPUT -p udp --dport 7844 -j REJECT
sudo iptables -A OUTPUT -p tcp --dport 7844 -j REJECT
# Watch the cloudflared terminal: connection registration failures.

# Prove the QUIC-only trap: block UDP alone and it falls back to HTTP/2.
sudo iptables -D OUTPUT -p tcp --dport 7844 -j REJECT
sudo pkill cloudflared
cloudflared tunnel --url http://localhost:8848 2>&1 | grep -i protocol
# → "Registered tunnel connection ... protocol=http2" ← still up

sudo iptables -F OUTPUT
```

WRITE THIS ONE DOWN

Blocking UDP/7844 alone does not stop the tunnel. It downgrades it to HTTP/2 over TCP/7844 and it keeps working. Any egress rule that only blocks QUIC gives you a false sense of coverage. Block both transports, for every provider.

9C — EXECUTION CONTROL

```
sudo chmod 000 /usr/local/bin/cloudflared
cloudflared tunnel --url http://localhost:8848      # Permission denied
sudo chmod 755 /usr/local/bin/cloudflared

# The real control — deny execution from user-writable paths
sudo dnf install -y fapolicyd 2>/dev/null || sudo apt-get install -y fapolicyd
echo 'deny_audit perm=execute all : dir=/home/' | \
  sudo tee /etc/fapolicyd/rules.d/25-mcp-tunnel.rules
sudo fagenrules --load && sudo systemctl restart fapolicyd
```

```
cp /usr/local/bin/cloudflared ~/cloudflared && ~/cloudflared --version # denied
sudo journalctl -u fapolicyd -n 20
```

9D — SERVER-SIDE PORTAL ENFORCEMENT

```
# Add the arrived_via_portal() check from Section 8.4 to mcp_lab_server.py,
# restart, then retry through the tunnel:
python3 mcp_probe.py "$TUNNEL_URL/mcp" call get_balance account_id=ACCT-1001
# → HTTP 400 "Direct access denied; use the MCP portal"
```

Result: the tunnel still exists and still forwards traffic, but the server refuses to act. This is the control that survives an end user switching clients, disabling a hook, or working off-network. It is the last line and the most durable one.

Step 10 The Windows run

POWERSHELL — SET UP

```
New-Item -ItemType Directory -Force C:\mcp-lab | Set-Location
# copy mcp_lab_server.py, mcp_probe.py, Find-McpTunnel.ps1 here

Invoke-WebRequest -Uri `
  "https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-windows-amd64.exe" `
  -OutFile C:\mcp-lab\cloudflared.exe

Start-Process python -ArgumentList `
  "C:\mcp-lab\mcp_lab_server.py --port 8848 --audit C:\mcp-lab\audit.jsonl" `
  -WindowStyle Hidden

Get-NetTCPConnection -State Listen -LocalPort 8848 |
  Select-Object LocalAddress, LocalPort, OwningProcess,
    @{n='Proc';e={{(Get-Process -Id $_.OwningProcess).ProcessName}}
```

POWERSHELL — TUNNEL AND OBSERVE

```
Start-Process C:\mcp-lab\cloudflared.exe `
  -ArgumentList "tunnel --url http://localhost:8848" `
  -RedirectStandardError C:\mcp-lab\cf.log -WindowStyle Hidden
Start-Sleep 8
$TunnelUrl = (Select-String -Path C:\mcp-lab\cf.log `
  -Pattern 'https://[a-z0-9-]+\+.trycloudflare\.com').Matches[0].Value
$TunnelUrl

# Confirm: outbound only. No new inbound listener on a routable address.
Get-NetTCPConnection -State Established |
  Where-Object { (Get-Process -Id $_.OwningProcess).ProcessName -eq 'cloudflared' } |
  Select-Object LocalAddress, LocalPort, RemoteAddress, RemotePort

# Invoke a tool through the public URL
$body = @{ jsonrpc='2.0'; id=1; method='tools/call'
  params=@{ name='get_balance'; arguments=@{ account_id='ACCT-1001' } } } |
  ConvertTo-Json -Depth 6
Invoke-RestMethod -Uri "$TunnelUrl/mcp" -Method Post -Body $body -ContentType 'application/json' `
  -Headers @{ 'MCP-Protocol-Version'='2026-07-28'
    'Mcp-Method'='tools/call'; 'Mcp-Name'='get_balance' }
```

POWERSHELL — READ THE SYSMON TELEMETRY

```
# Event 1 — tunnel agent execution
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; ID=1} `
  -MaxEvents 300 |
```

```
Where-Object { $_.Message -match 'cloudflared|ngrok|devtunnel|mcp-remote|supergateway' } |
ForEach-Object {
    [pscustomobject]@{
        Time = $_.TimeCreated
        Cmd = ([regex]'CommandLine: (.+)').Match($_.Message).Groups[1].Value
        Parent = ([regex]'ParentImage: (.+)').Match($_.Message).Groups[1].Value
    }
} | Format-Table -Wrap

# Event 22 – DNS to provider infrastructure
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; ID=22} `
    -MaxEvents 500 |
Where-Object { $_.Message -match 'argotunnel|trycloudflare|ngrok|devtunnels' } |
Select-Object TimeCreated, @{n='Query';e={
    ([regex]'QueryName: (.+)').Match($_.Message).Groups[1].Value }}

# Full triage, including the loopback + tunnel correlation
.\Find-McpTunnel.ps1
.\Find-McpTunnel.ps1 -AsJson | Out-File C:\mcp-lab\trriage.json
```

POWERSHELL — WDAC / APPLOCKER TEST AND TEARDOWN

```
# Confirm enforcement mode before you conclude anything about coverage
Get-AppLockerPolicy -Effective -Xml | Select-String 'EnforcementMode'
Test-AppLockerPolicy -Policy (Get-AppLockerPolicy -Effective) `
    -Path C:\mcp-lab\cloudflared.exe -User Everyone

# Teardown
Get-Process cloudflared -ErrorAction SilentlyContinue | Stop-Process -Force
Get-Process python -ErrorAction SilentlyContinue |
    Where-Object { $_.CommandLine -match 'mcp_lab_server' } | Stop-Process -Force
Remove-Item C:\mcp-lab\cloudflared.exe -Force
```

Step 11 Teardown and artifact checklist

EC2 TEARDOWN

```
sudo pkill cloudflared; pkill -f mcp_lab_server.py; sudo pkill tcpdump
sudo iptables -F OUTPUT
sudo rm -f /etc/audit/rules.d/99-mcp-tunnel.rules /etc/fapolicyd/rules.d/25-mcp-tunnel.rules
sudo augenrules --load; sudo systemctl restart auditd
sudo rm -f /usr/local/bin/cloudflared
[ -f /etc/hosts.bak ] && sudo mv /etc/hosts.bak /etc/hosts
ss -tulnp; ps aux | grep -Ei 'cloudflared|mcp_lab' | grep -v grep
# then terminate the instance
```

Artifacts to keep for the report

Artifact	What it evidences
scan-before.txt + post-tunnel nmap	Identical clean scans before and after the service became internet-reachable. The headline exhibit.
Security group JSON	Zero relevant inbound rules, unchanged throughout.
socket-timeline.txt	Only outbound ESTABLISHED sessions appear. No new listener on a routable address.
cloudflared.log	Timestamp of publication and the assigned public hostname — and the protocol=http2 line from step 9b.

Artifact	What it evidences
mcp-audit.jsonl	An external caller invoking a write-critical tool, with Cf-Ray and X-Forwarded-For present.
mitmproxy extract	Tool name recoverable with decryption; opaque without it.
Step 9 results table	Which controls worked, which downgraded, which failed. The basis for your recommendation.

10. Control matrix

Maps each control to the tunnel types it addresses, the authoritative source that recommends it, and the lab step that validates it. Use this as the appendix to a control standard.

Control	Types	OWASP	Authority	Lab	Evidence of effectiveness
Application allowlisting for tunnel binaries	T1 T2	MCP09	NSA CSI (constrain and sandbox)	9c	Tunnel cannot start. Root-cause control.
Deny execution from user-writable paths	T1 T2	MCP04 MCP09	NSA CSI	9c	Blocks the entire class including unknown tools.
Default-deny egress via TLS-inspecting proxy	T1 T2 T3	MCP10	NSA CSI (filtering outbound proxy / DLP)	8, 9b	Recovers tool name and arguments; blocks unapproved destinations.
Block UDP/443 and UDP/7844 outbound	T2	—	Lab finding	9b	Required or QUIC bypasses the proxy. UDP-only blocking downgrades, does not stop.
DNS policy on provider zones	T2	MCP09	Sigma / vendor detections	9a	Prevents new tunnels. Does not terminate live ones.
MCP classification at the gateway	T2 T3 T5	MCP08 MCP09	Cloudflare, Aug 2026	8	Header-based, hostname-independent. Needs decryption.
Portal-only enforcement at the network	T5	MCP07	Cloudflare Traffic Source	—	Blocks approved-server bypass on managed paths only.
Origin-side rejection of non-portal traffic	T2 T5	MCP07	MCP spec; WriteGuard pattern	9d	Survives client switching and off-network use. Most durable.
Risk-tiered server-side tool authorization	T3 T4 T5	MCP02 MCP07	NSA CSI; WriteGuard	2, 9d	Last point of denial before execution.
Structured MCP audit logging to SIEM	All	MCP08	NSA CSI (explicit)	2, 6, 7	Only source with tool-level semantics. Sole answer to T4.
Token audience validation; no passthrough	T3 T5	MCP01 MCP07	MCP spec (forbidden)	—	Closes confused deputy across the tool boundary.
Pre-registered OAuth clients; no DCR; RFC 9207 iss	T3 T5	MCP01 MCP07	MCP 2026-07-28	—	Closes the AS mix-up and DCR consent-bypass paths.
Tool-description hash pinning and drift alerts	T3 T4	MCP03	Trail of Bits TOFU pinning	7.8	Detects rug-pull changes to tool metadata.
Client config inventory and change detection	T1 T3	MCP09	Multi-layer discovery practice	6.2, 7.2	Only reliable inventory of stdio servers.
Enterprise-managed client policy	T1 T3	MCP09	Vendor managed settings	—	Covers stdio, which no network control reaches.

Control	Types	OWASP	Authority	Lab	Evidence of effectiveness
Loopback listener + tunnel process correlation	T2	MCP09	This guide, §7.7	7, 10	Highest-precision detection available at the host.
Human-in-the-loop for money movement	All	MCP02	NSA CSI; platform guidance	—	Bounds worst-case impact regardless of detection failure.
Network scan for exposed MCP servers	T3	MCP09	NSA CSI (explicit)	1, 3	Catches routable-bound servers. Misses every tunnelled loopback server.

11. Assessment checklist for the copilot programme

Work through this per agent or per tool surface. "Unknown" is a finding, not a blank.

Inventory and governance

- Can you enumerate every MCP server reachable from every copilot, agent and developer client, on demand? Include stdio servers, project-scoped `.mcp.json` in repositories, and CI-only servers.
- Is there an approval gate before a new MCP server is added to any client, and is it enforced technically rather than by policy alone?
- Do you have an AI Bill of Materials that lists MCP servers, versions, and their tool catalogues?
- Are archived or unmaintained MCP server projects flagged? The MCP project itself maintains a list of archived servers, some of which touch filesystems and developer tooling.

The trifecta test, per agent

- Does this agent access sensitive or regulated data? (*yes / no*)
- Is it exposed to content the bank does not control — email, documents, web, tickets, supplier portals? (*yes / no*)
- Does it have an outbound channel — a write tool, an email action, a generic HTTP connector, a web-fetch tool? (*yes / no*)
- **Three yeses requires a documented compensating control and named risk acceptance.**

Money movement

- List every tool that can create or modify a beneficiary, initiate a transfer, or change payment instructions. Is each risk-tiered in its definition and enforced at the handler?
- Is maker-checker enforced in the core system, or only in the agent workflow?
- Are beneficiary details shown for confirmation re-read from the system of record, not echoed from agent context?
- Are idempotency keys mandatory on every state-changing tool?
- Are per-identity, per-tool velocity limits in place on `tools/call`?
- Worst single sequence of tool calls a fully compromised model could execute before a human sees anything — write it out.

Transport and tunnels

- Do you know which MCP protocol versions your clients speak? This determines achievable detection fidelity.
- Is TLS inspection enabled on paths carrying agent traffic? If not, MCP classification is unavailable to you.
- Are UDP/443 and UDP/7844 blocked outbound from agent-hosting subnets?
- Is execution of tunnel binaries prevented, or only detected? Does the policy cover `ssh -R` and self-hosted tools with no provider domain?
- Is there a governed MCP path, and do your own servers reject requests that bypass it?

- Have you run the loopback-listener plus tunnel-process correlation across your developer estate? What did it return?

Logging and response

- Do your MCP servers log tool name, caller identity, argument digest, risk tier and outcome? Does it reach the SIEM?
- Are tool arguments hashed rather than written in clear? Arguments routinely contain account numbers and PII.
- Is W3C Trace Context propagated so a tool call correlates from host application through gateway to server?
- Do you alert on header-versus-body method mismatches and on provider trace headers arriving at internal-only servers?
- Does the IR playbook cover an agentic incident: how to identify every tool call in a session, revoke the agent's credentials, and determine what data left?
- Are low-code platform injection-shield events actually arriving in the SIEM, verified by test?

Supply chain

- Is `mcp-remote` present anywhere, and is every instance at 0.1.16 or later? (CVE-2025-6514, CVSS 9.6)
- Is MCP Inspector present, and at 0.14.1 or later? (CVE-2025-49596)
- Are tool descriptions hash-pinned at approval and diffed on reconnect?
- Do you scan tool descriptions for invisible Unicode and instruction-injection phrasing?
- Is there a defined process for tracking MCP CVEs, as the NSA CSI recommends?

12. References

Retrieved 2 September 2026. Vendor endpoints and product behaviour change; re-verify provider domains and ports before deploying egress or DNS policy.

Specification and protocol

1. Model Context Protocol. *The 2026-07-28 Specification*. 28 July 2026. Stateless core, MRTR, header-based routing, authorization hardening, deprecations.
<https://blog.modelcontextprotocol.io/posts/2026-07-28/>
2. Model Context Protocol. *Security Best Practices* — confused deputy, token passthrough, session hijacking.
https://modelcontextprotocol.io/specification/latest/basic/security_best_practices
3. Model Context Protocol. *Transports* — stdio and Streamable HTTP.
<https://modelcontextprotocol.io/specification/2026-07-28/basic/transports>
4. Model Context Protocol. *The 2026 MCP Roadmap*. March 2026.
<https://blog.modelcontextprotocol.io/posts/2026-mcp-roadmap/>

Authoritative guidance

1. NSA Artificial Intelligence Security Center. *Model Context Protocol (MCP): Security Design Considerations for AI-Driven Automation*. CSI U/OO/6030316-26, PP-26-1834, May 2026 v1.0. With CMU SEI.
https://media.defense.gov/2026/Jun/02/2003943289/-1/-1/0/CSI_MCP_SECURITY.PDF
2. OWASP. *MCP Top 10 (2025)*. Including MCP09 Shadow MCP Servers and MCP03 Tool Poisoning.
<https://owasp.org/www-project-mcp-top-10/>
3. Equixly. *Mapping NSA's MCP guidance to the OWASP MCP Top 10: How to test for the risks*. June 2026.
<https://equixly.com/blog/2026/06/04/mapping-nsa-s-mcp-guidance-to-the-owasp-mcp-top-10-how-to-test-for-the-risks/>

Detection and network control

1. Cloudflare. *How Cloudflare detects MCP traffic and helps secure it*. 14 August 2026. Anatomy of a tool call; three control points; `experimental.is_mcp`; Traffic Source selectors.
<https://blog.cloudflare.com/mcp-security-updates/>
2. Cloudflare. *Detect MCP traffic in Gateway logs* — GraphQL queries and DLP profiles for JSON-RPC methods.
<https://developers.cloudflare.com/cloudflare-one/tutorials/detect-mcp-traffic-gateway-logs/>
3. SigmaHQ. *Cloudflared Quick Tunnel Execution (222129f7-f4dc-4568-b0d2-22440a9639ba); Network Connection Initiated To Cloudflared Tunnels Domains; PUA - Ngrok Execution (ee37eb7c-a4e7-4cd5-8fa4-efa27f1c3f31)*.
<https://github.com/SigmaHQ/sigma>
4. Elastic. *Potential Protocol Tunneling via Cloudflared* — prebuilt detection rule and response guidance.
https://www.elastic.co/docs/reference/security/prebuilt-rules/rules/windows/command_and_control_tunnel_cloudflared
5. MITRE ATT&CK: T1572 Protocol Tunneling; T1567 Exfiltration Over Web Service; T1090 Proxy.
<https://attack.mitre.org/techniques/T1572/>

Vulnerabilities and incidents

1. JFrog Security Research. *CVE-2025-6514: Critical RCE in mcp-remote*. CVSS 9.6. Affects 0.0.5–0.1.15; fixed 0.1.16.
<https://jfrog.com/blog/2025-6514-critical-mcp-remote-rce-vulnerability/>
2. Oligo Security. *Critical RCE in Anthropic MCP Inspector — CVE-2025-49596*. Fixed 0.14.1.
<https://www.oligo.security/blog/critical-rce-vulnerability-in-anthropic-mcp-inspector-cve-2025-49596>
3. Aim Security. *EchoLeak — CVE-2025-32711*, zero-click exfiltration from M365 Copilot, CVSS 9.3.
4. Invariant Labs. *GitHub MCP Exploited: accessing private repositories via MCP; WhatsApp MCP Exploited; Toxic Flows in Agentic Systems*.
<https://invariantlabs.ai/blog/mcp-github-vulnerability>

5. Cloud Security Alliance Labs. *Poisoned MCP Tool Descriptions: A Silent Exfiltration Path*. July 2026.
<https://labs.cloudsecurityalliance.org/research/csa-research-note-mcp-tool-description-poisoning-20260711-cs/>

Research

1. *Hiding in the AI Traffic: Abusing MCP for LLM-Powered Agentic Red Teaming*. arXiv:2511.15998, November 2025. MCP as a covert C2 substrate; elimination of beacon periodicity.
<https://arxiv.org/abs/2511.15998>
2. Radosevich & Halloran. *MCP Safety Audit: LLMs with the Model Context Protocol Allow Major Security Exploits*. arXiv: 2504.03767.
<https://arxiv.org/abs/2504.03767>
3. Zhao et al. *Mind Your Server: A Systematic Study of Parasitic Toolchain Attacks on the MCP Ecosystem*. arXiv: 2509.06572.
<https://arxiv.org/abs/2509.06572>
4. HiddenLayer. *Exploiting MCP Tool Parameters*. 2025.
<https://hiddenlayer.com/innovation-hub/exploiting-mcp-tool-parameters/>

Platform and low-code

1. BeyondTrust Phantom Labs. *A Security Researcher's Guide to Understanding Copilot Studio AI Agents*. May 2026.
<https://www.beyondtrust.com/blog/entry/copilot-studio-ai-agents-security-risks>
2. Microsoft. *Protecting against indirect prompt injection attacks in MCP*.
<https://developer.microsoft.com/blog/protecting-against-indirect-injection-attacks-mcp>
3. Microsoft. *MCP Security Best Practices* (mcp-for-beginners, Security module).
<https://github.com/microsoft/mcp-for-beginners>

COMPANION FILES

`mcp_lab_server.py` — stdlib-only spec-conformant MCP server with reference audit logging.
`mcp_probe.py` — stdlib-only MCP client for generating conformant and non-conformant wire traffic.
`mcp_hunt.sh` — Linux host triage with the loopback-plus-tunnel correlation.
`Find-McpTunnel.ps1` — Windows equivalent, with JSON output for SIEM ingestion.